



计算机网络

第四章 端到端传输协议

徐敬东 张建忠

xujd@nankai.edu.cn

zhangjz@nankai.edu.cn

计算机网络与信息安全研究室

学习目标



总体目标：理解端到端传输协议需要解决的基本问题，以及具体的工作机制和优化策略

了解传输层的作用，理解用户数据报协议UDP的基本工作原理和所提供的服务

掌握TCP协议的连接管理、可靠数据传输、流量控制、拥塞控制的实现原理，以及TCP服务

理解TCP协议所面临的问题，初步了解MTCP、QUIC等扩展和替代方案的工作机制

培养思辨能力和创新能力，掌握可靠传输协议的设计方法，培养较强的网络编程能力

4.1 用户数据报协议

4.1.2 多路复用和分用

4.1.3 差错检测

4.2 传输控制协议

4.3 TCP扩展与替代协议

4.4 实时传输协议

4.1 用户数据报协议



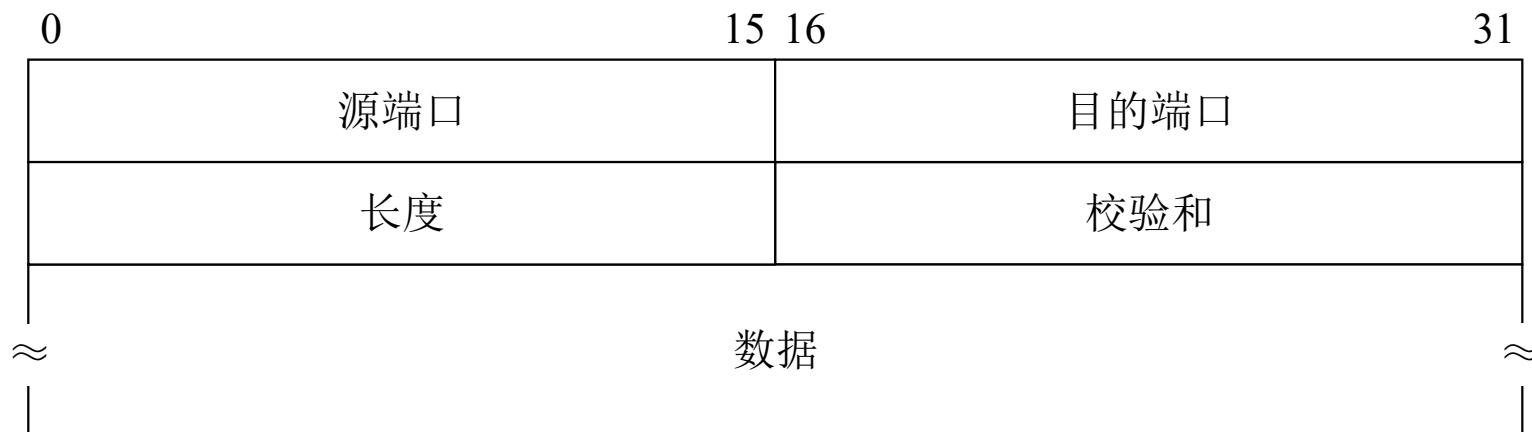
- UDP是一个轻量级的端到端传输协议
- UDP运行在互联层之上，主要实现多路复用和分用功能和简单的差错检测功能
- UDP向上层提供面向无连接、不可靠的传输服务
 - 无需建立连接，UDP数据报之间相互独立，后发的数据报可能会早于先发的数据报到达接收方，接收方按照数据报到达的顺序进行处理，并将数据提交给上层
 - UDP服务不具有可靠性，数据报在传输过程中可能会出现差错或丢失，UDP不提供数据报恢复功能，使用UDP服务的应用需要自行解决可靠性问题

4.1 用户数据报协议



4.1.1 UDP数据报

- UDP的协议数据单元称为**UDP数据报**
- **UDP数据报**由8字节的首部和可变长度的数据部分组成
 - **源端口与目的端口**：标识发送进程和接收进程，16位端口可以区分一台主机中的65536个进程
 - **长度**：指明包括首部在内的UDP数据报的长度，计数单位为字节
 - **校验和**：对UDP伪首部和UDP数据报进行校验



4.1 用户数据报协议



4.1.2 多路复用和分用

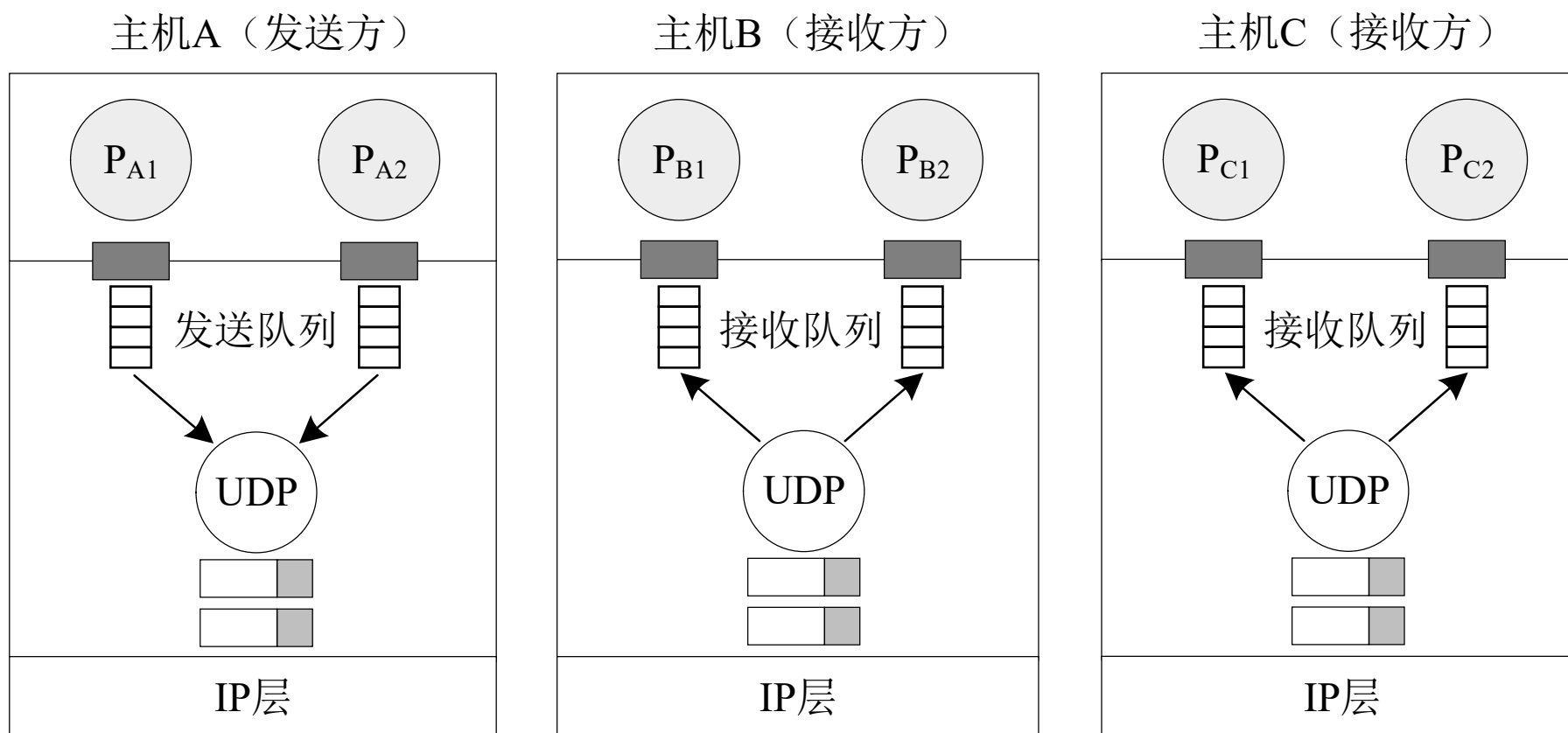
- UDP使用端口标识一台主机中运行的应用进程，相同主机中的不同应用进程需要使用不同的端口进行标识
 - **多路复用**：从应用进程接收数据，增加UDP首部（包含源端口和目的端口），组成UDP数据报，并通过互联层进行发送
 - **多路分用**：对到达的UDP数据报进行解析，根据目的端口，将数据写入对应的接收缓冲区
- 每台主机中的UDP端口是独立分配的，不同主机的应用进程可能会使用相同的端口，因而需要将主机寻址与UDP端口相结合来唯一标识一个使用UDP服务的应用进程
 - 进程标识：**IP地址+UDP端口（二元组）**

4.1 用户数据报协议



4.1.2 多路复用和分用

■ UDP复用和分用示例



4.1 用户数据报协议



4.1.3 差错检测

- UDP差错检测范围：UDP伪首部和整个UDP数据报
- **发送方**：利用自己产生的伪首部和待发送的UDP数据报计算校验和
- **接收方**：利用自己产生的伪首部和接收到的UDP数据报计算校验和
- **伪首部格式**：包含源IP地址、目的IP地址、协议类型（17）、长度等，长度与UDP数据报的长度字段相同

0	15	16	31
源IP地址			
目的IP地址			
0	协议类型		长度

4.1 用户数据报协议



4.1.3 差错检测

发送方

- 校验和字段置0，数据报用0补齐为16位的倍数
- 产生伪首部，将伪首部和UDP数据报一起视为16位二进制整数序列
- 进行16位二进制求和运算，结果取反写入校验和字段

接收方

- 重新产生伪首部，并将数据报用0补齐为16位的倍数
- 对伪首部和UDP数据报进行16位二进制求和运算
- 计算结果为全1，没有检测到错误；否则，数据报存在差错

注：UDP对存在差错的数据报做丢弃处理

4.1 用户数据报协议



4.1.3 差错检测

■ 校验和计算示例

伪首部	202.113.10.33			
	202.113.20.18			
首部	0	17	13	
	2096		53	
数据	13		0	
	10010011	00010011	10110010	00011111
	11110011	0填充		

11001010 01110001 (202.113)

00001010 00100001 (10.33)

11001010 01110001 (202.113)

00010100 00010010 (20.18)

00000000 00010001 (0+17)

00000000 00001101 (13)

00001000 00110000 (2096)

00000000 00110101 (53)

00000000 00001101 (13)

00000000 00000000 (0)

10010011 00010011 (数据)

10110010 00011111 (数据)

11110011 00000000 (数据+填充)

11110011 11011010 (求和结果)

00001100 00100101 (取反后的校验和)

4.1.3 差错检测

■ UDP校验和计算几点说明

- IPv4之上UDP的校验和计算是可选项
 - 如果校验和字段为0，代表发送方未进行差错校验
 - 如果校验和字段为非0，代表发送方进行了差错校验
 - 如果发送方计算的校验和为0，则以全1代替
- UDP校验和的计算覆盖UDP伪首部和UDP数据报，使用伪首部的目的是检验UDP数据报是否到达了正确的目的主机
- 伪首部和0填充部分不随用户数据报一起传输，接收方需自己形成伪首部进行校验
- 伪首部的使用破坏了层次划分
 - 目的主机的IP地址UDP通常知道，源IP的使用需要通过路由选择决定

4.1 用户数据报协议

4.2 传输控制协议

4.2.1 TCP段

4.2.2 TCP连接管理

4.2.3 TCP可靠数据传输

4.2.4 TCP拥塞控制

4.3 TCP扩展与替代协议

4.4 实时传输协议

4.2 传输控制协议



- TCP运行在互连层之，主要功能包括
 - 多路复用和分用
 - 可靠数据传输
 - 流量控制
 - 拥塞控制
- TCP向上层提供**面向连接的、可靠的字节流服务**
 - 面向连接：发送数据之前发送方和接收方之间需要建立连接
 - 可靠字节流服务
 - 将上层数据视为字节流，不识别报文边界
 - 提供按序、可靠交付
- TCP只提供点对点通信

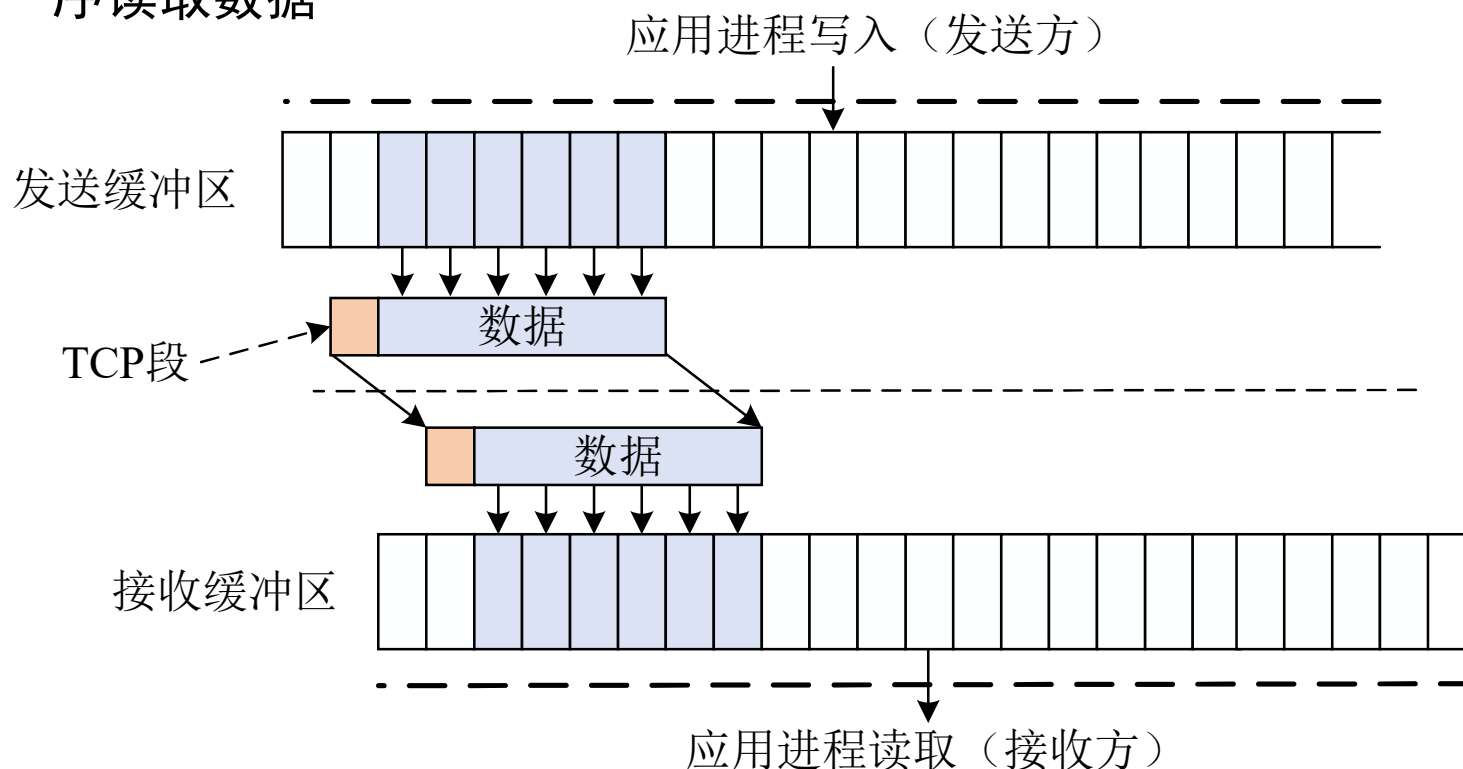
4.2 传输控制协议



4.2.1 TCP段

■ 面向连接的字节流服务与TCP段

- 发送方应用进程将数据以字节流的方式写入发送缓冲区，不区分报文边界，TCP利用TCP段承载数据，接收方应用进程从接收缓冲区中按序读取数据



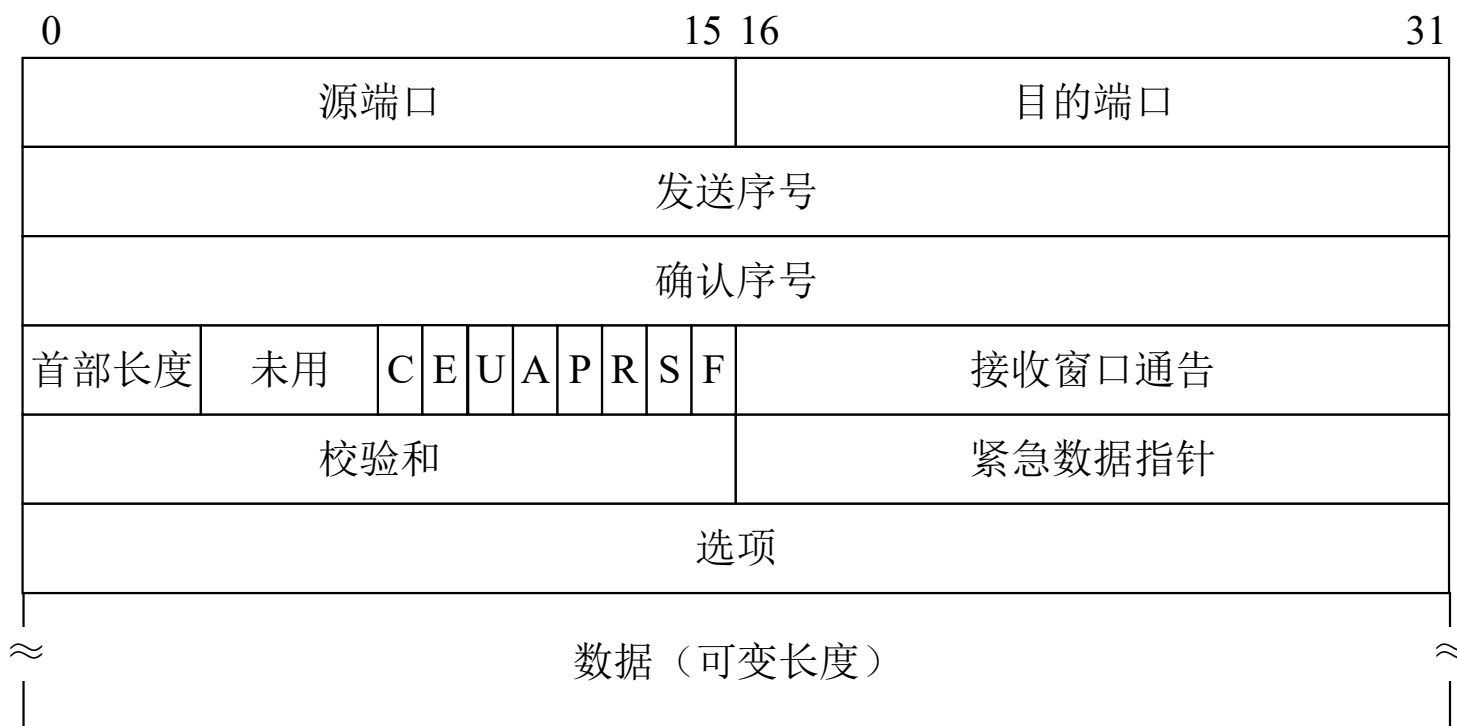
4.2 传输控制协议



4.2.1 TCP段

■ 段格式：首部+可变长度数据

- 首部：由20字节的固定部分和可变长度的选项部分组成



首部长度：以4字节为计数单位，包含选项部分

标志位：CWR, ECE, URG, ACK, PUSH, RESET, SYN, FIN

4.2 传输控制协议



4.2.1 TCP段

■ 段格式：选项

- TCP段首部最多可以包含40字节的选项
- **单字节选项**：包含选项结束和无操作
- **多字节选项**：包含最大段长度、窗口扩展因子、时间戳、选择确认选项等
- **多字节选项格式**：由**类型**、**长度**和**信息**3部分组成，其中长度字段指明整个选项的字节数

类型（1字节）	长度（1字节）	信息（可变）
---------	---------	--------

4.2 传输控制协议



4.2.2 TCP连接管理

- TCP提供面向连接的字节流服务，在使用TCP服务时首先要建立连接，并在数据传输结束时关闭连接
- TCP连接由<源IP、源端口、目的IP、目的端口>四元组唯一标识
- TCP连接的端点可以抽象为一对套接字，用于标识一对利用该TCP连接进行通信的应用进程
- TCP通过三次握手建立连接，通过四次挥手关闭连接

4.2 传输控制协议



4.2.2 TCP连接管理

■ TCP连接建立

➤ 建立连接的目的

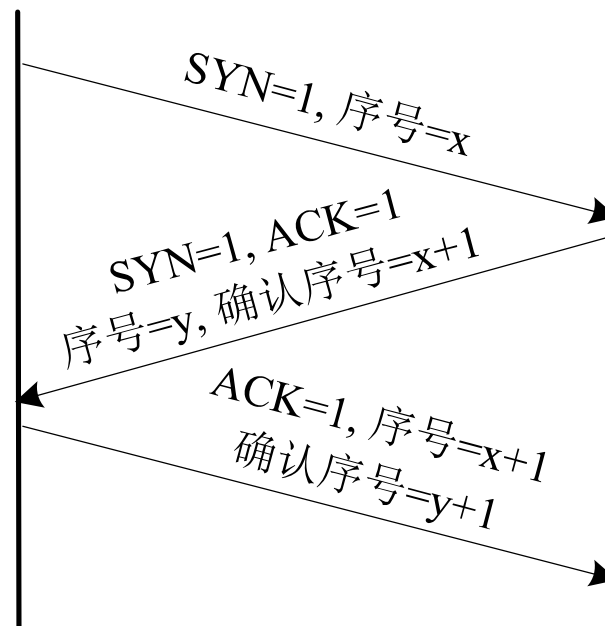
- 确认对方的应用进程是否在特定的端口上运行
- 通告商定参数, 如最大段长度(MSS), 初始序号(ISN)、接收窗口大小等

➤ 三次握手建立连接

- 步骤1: A发送SYN段, **分配缓冲区**, 选择A→B初始序号
- 步骤2: B接收SYN段, 回送SYN+ACK段, **分配缓冲区**, 选择B→A的初始序号
- 步骤3: A回送ACK, **可以同时发送数据**

主机 A
(主动打开)

主机 B
(被动打开)



SYN段: SYN标志置位的TCP段 **ACK段:** ACK标志置位的TCP段

4.2.2 TCP连接管理

■ TCP连接建立

- 双向数据流的ISN由各自的发送方确定，通常不是一个固定值，生成方式依赖于操作系统的具体实现
 - **RFC 793的建议方法**：每个TCP实体维护一个32位计数器，该计数器每4微秒增1，建立连接时从中读取计数器当前值作为ISN
 - **RFC 6528的生成方法**：利用本地IP、本地端口、远端IP、远端端口等，通过哈希运算产生一个伪随机数，与计数器中的当前值求和
- 生成随机ISN的目的
 - 避免使用相同套接字的新旧TCP连接中TCP段的混淆
 - 可以避免攻击者猜测某个TCP连接的ISN

4.2 传输控制协议



4.2.2 TCP连接管理

■ TCP连接建立

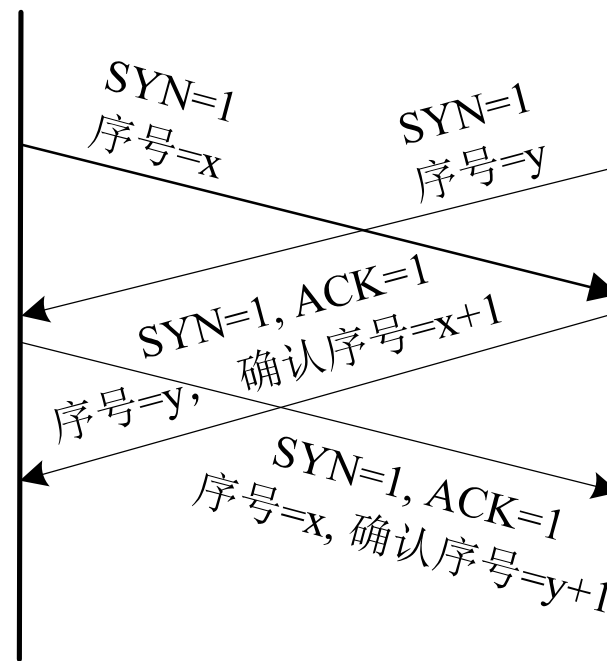
➤ **同时打开**：TCP允许双方同时执行主动打开。但在实际中，这种情况发生的可能性很小

➤ **连接重试**

- 如果一方发出的连接请求在设定的时间内得不到另一方的确认，则会重传连接请求
- 重传的时间间隔设定与具体实现有关。例如，Windows10的重传间隔分别约为1秒、2秒、4秒、8秒，呈指数增长。4次重传后仍未得到确认，则放弃TCP连接请求

主机 A
(主动打开)

主机 B
(主动打开)



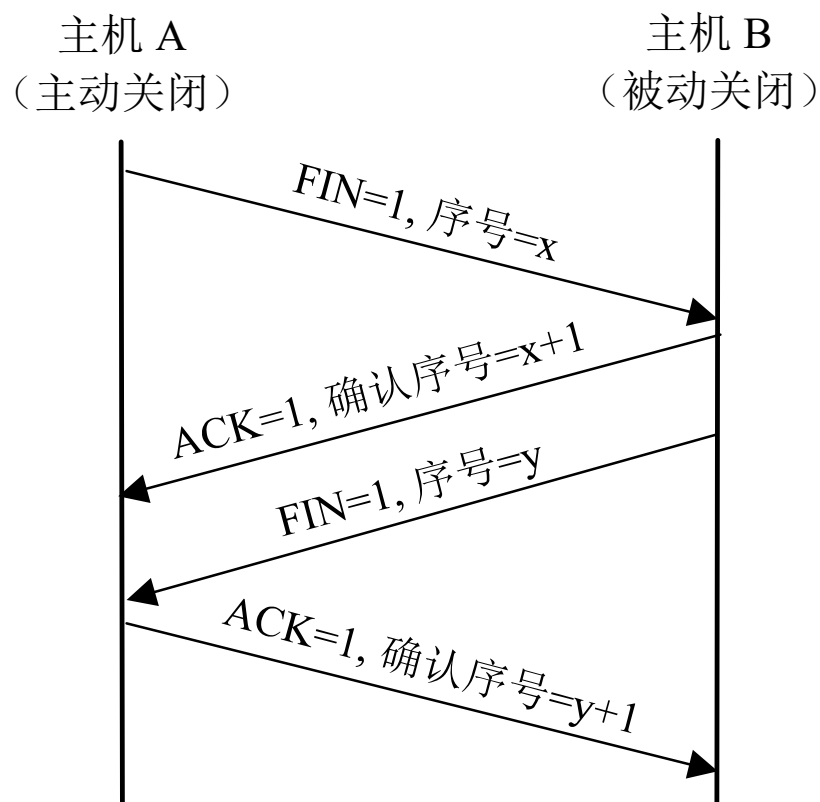
4.2 传输控制协议



4.2.2 TCP连接管理

■ TCP连接关闭

- 步骤1: A发送FIN段, A不再向B发送数据, 但仍可以接收数据
- 步骤2: B接收FIN段, 回送ACK段, B仍可以向A发送数据
- 步骤3: B发送FIN段, 等待A返回ACK
- 步骤4: A接收FIN段, 回送ACK段, 等待两倍的段生存期关闭连接; B接收ACK, 关闭连接



FIN段: FIN标志置位的TCP段

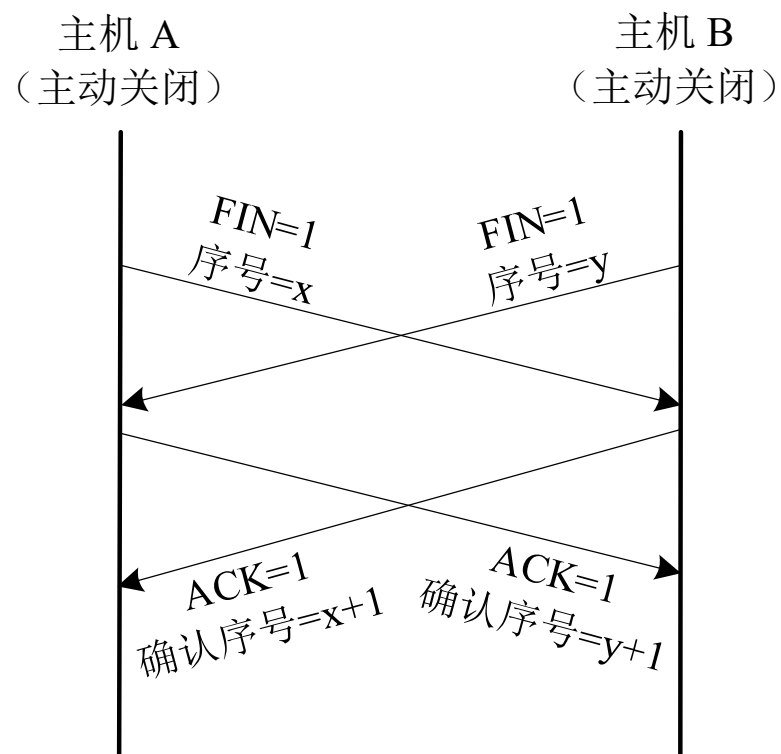
4.2 传输控制协议



4.2.2 TCP连接管理

■ TCP连接关闭

➤ **同时关闭**：TCP允许双方同时关闭TCP连接接



4.2.2 TCP连接管理

■ 复位段的使用

- **复位段**：RESET (RST) 标志位置位的TCP段
- 复位段主要用于处理TCP连接中的一些异常情况，目前也被用于快速关闭TCP连接
- 接收到复位段的一方不需要给出确认，直接终止连接

■ **半打开连接**：连接的一方已经关闭或异常终止连接而未告知另一方，另一方还处在正常的连接状态

- 例如，在客户-服务器交互模式中，由于客户的非正常关机，会使服务器产生较多的半打开连接
- 可以通过**保活定时器**及**发送空闲探测报文段**来识别半打开连接，使用复位段终止TCP连接

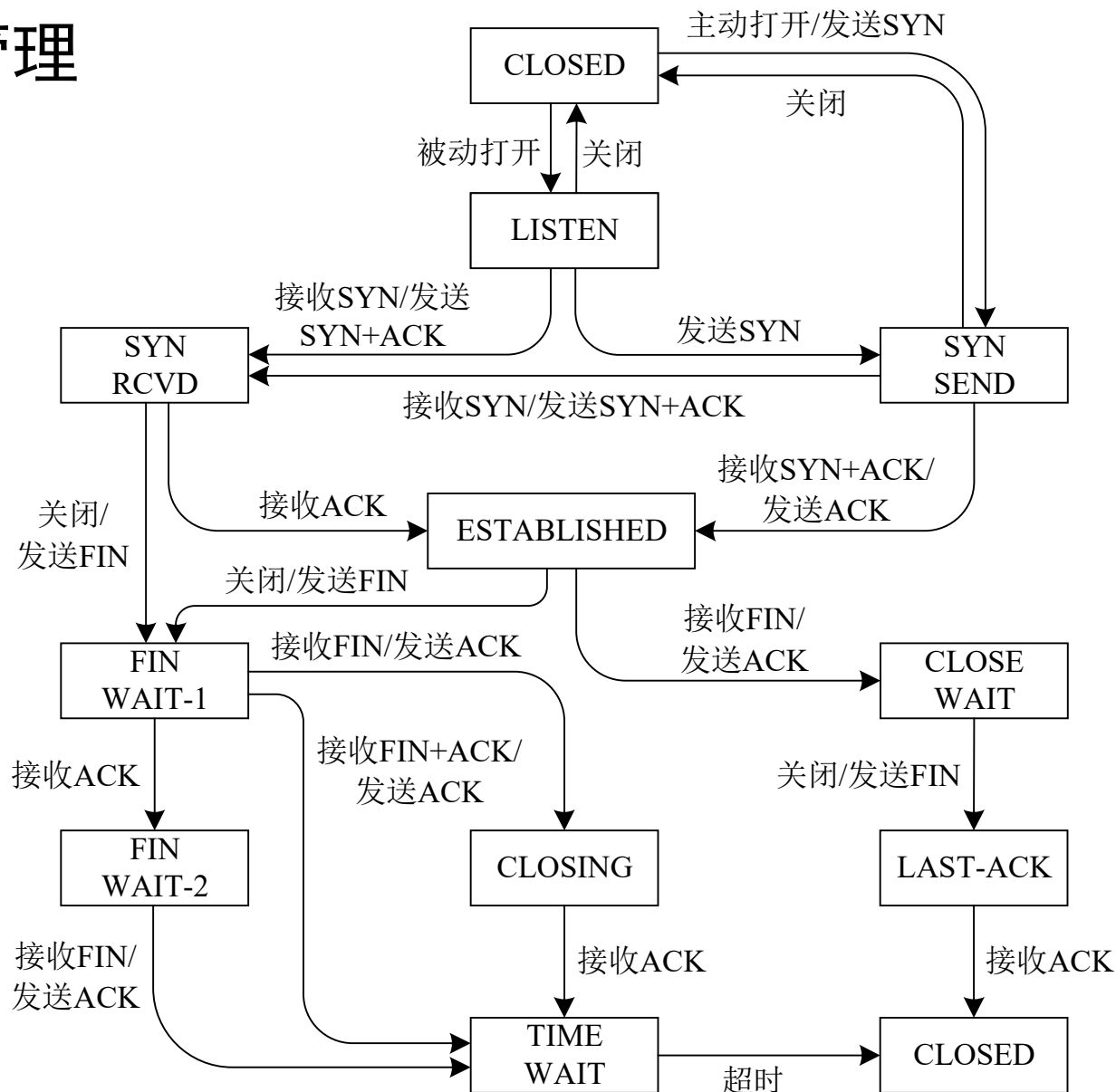
4.2 传输控制协议



4.2.2 TCP连接管理

■ TCP状态图

触发状态转移的事件包括：来自对等实体的TCP段或来自本地应用进程的调用



4.2 传输控制协议

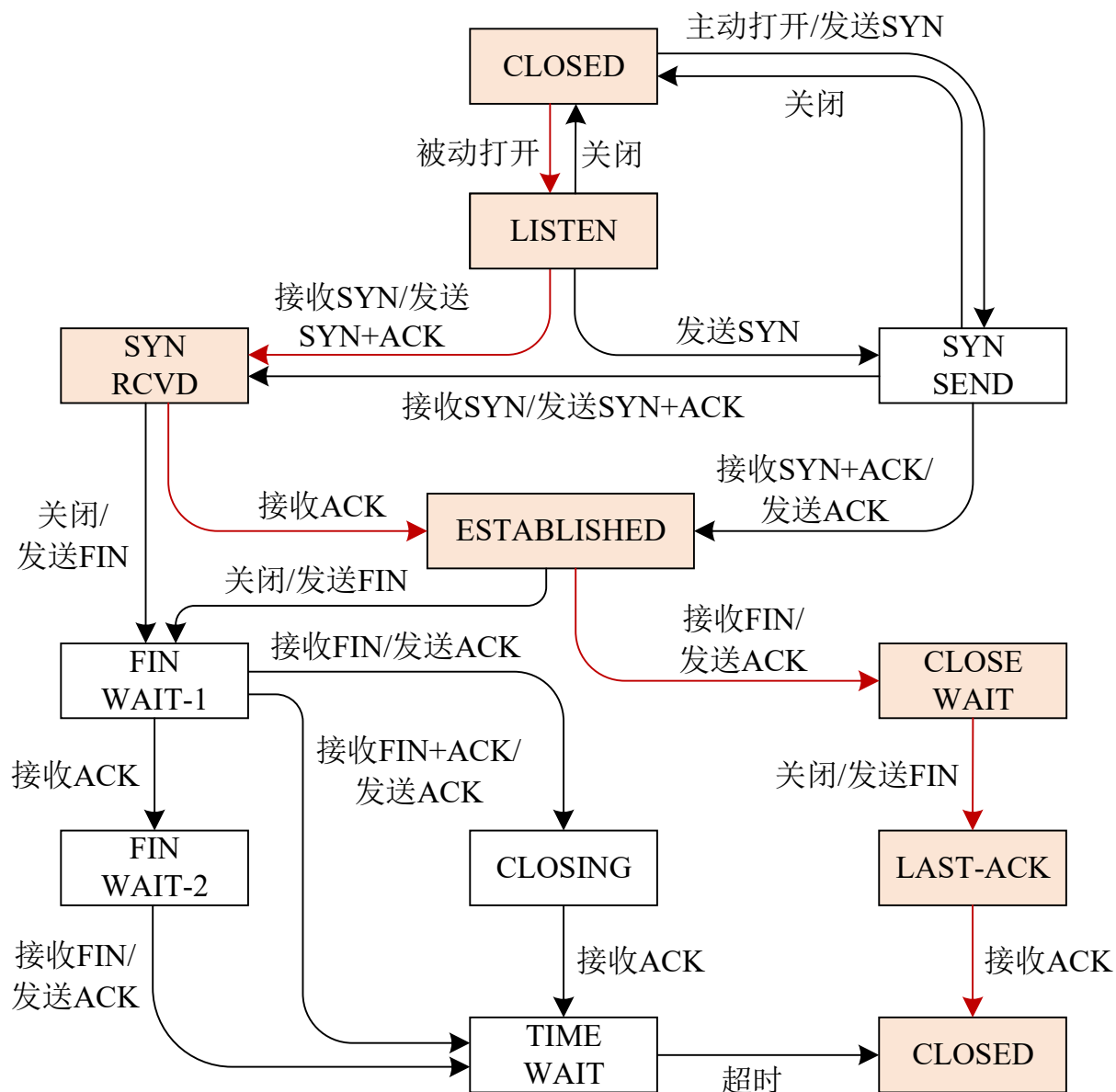


4.2.2 TCP连接管理

■ TCP状态图

- **建连**：服务器进程被动打开，客户进程主动打开
- **关闭**：客户进程首先发起连接关闭请求，服务器之后关闭连接

服务器端TCP
状态转移过程



4.2 传输控制协议

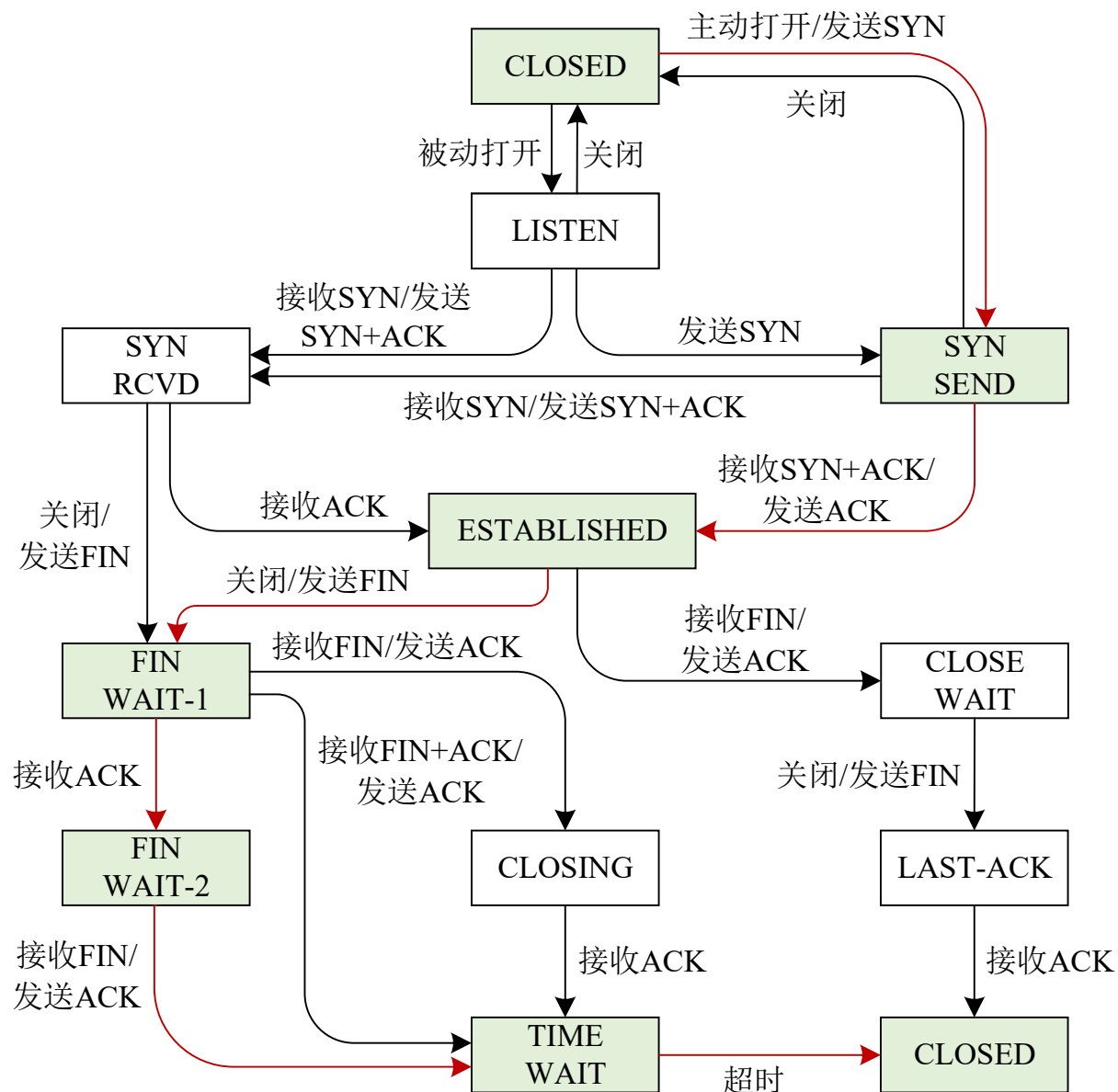


4.2.2 TCP连接管理

■ TCP状态图

- **建连**：服务器进程被动打开，客户进程主动打开
- **关闭**：客户进程首先发起连接关闭请求，服务器之后关闭连接

客户端TCP
状态转移过程



4.2 传输控制协议



4.2.3 TCP可靠数据传输

- **目标**：将发送缓冲区中的数据按序、正确地传送到接收缓冲区，保证接收到的字节流和发送的字节流完全一致
- **解决的问题**：出错、丢失、重复、乱序等
 - 基于自动重传请求（ARQ）实现可靠传输
 - 通过确认和超时重传机制恢复出错和丢失的TCP段，利用序号判断重复的数据，并保证数据的顺序性
- **适用的应用**：主要面向对**可靠性要求高**、对**传输延时不十分敏感**的应用，不适合对实时性要求较高的交互式应用，特别是实时多媒体应用

4.2 传输控制协议



4.2.3 TCP可靠数据传输

■ TCP段发送时机

- **问题**：在一个TCP连接上，发送方的TCP实体将应用进程写入到发送缓冲区的数据视为字节流，不区分应用层报文的边界，**如何确定何时发送一个TCP段**
- 封装并发送TCP段时机：
 - 当发送缓冲区中的数据达到最大段长度（MSS）时
 - 发送方的应用进程指明要求发送TCP段，即PUSH操作
 - TCP实体维护的定时器超时

注：TCP支持双向数据流，但是便于理解，这里只考虑数据单向流动的场景，即数据从发送方流向接收方，接收方给出确认

4.2 传输控制协议



4.2.3 TCP可靠数据传输

■ 最大段长度（MSS）

- MSS为每个TCP段所能承载数据部分的最大长度，不包括TCP首部
- MSS可以在建立连接时通过TCP选项进行协商，最大可达65535字节
- 如果一方不接受另一方通告的MSS值，则使用默认的536字节
- MSS受限于物理网络的MTU，尽可能保证IP数据包不分片
 - 例如，以太网的MTU为1500字节，减去20字节的IP首部和20字节的TCP首部，MSS可设置成1460字节

类型=2	长度=4	最大段长度（2字节）
------	------	------------

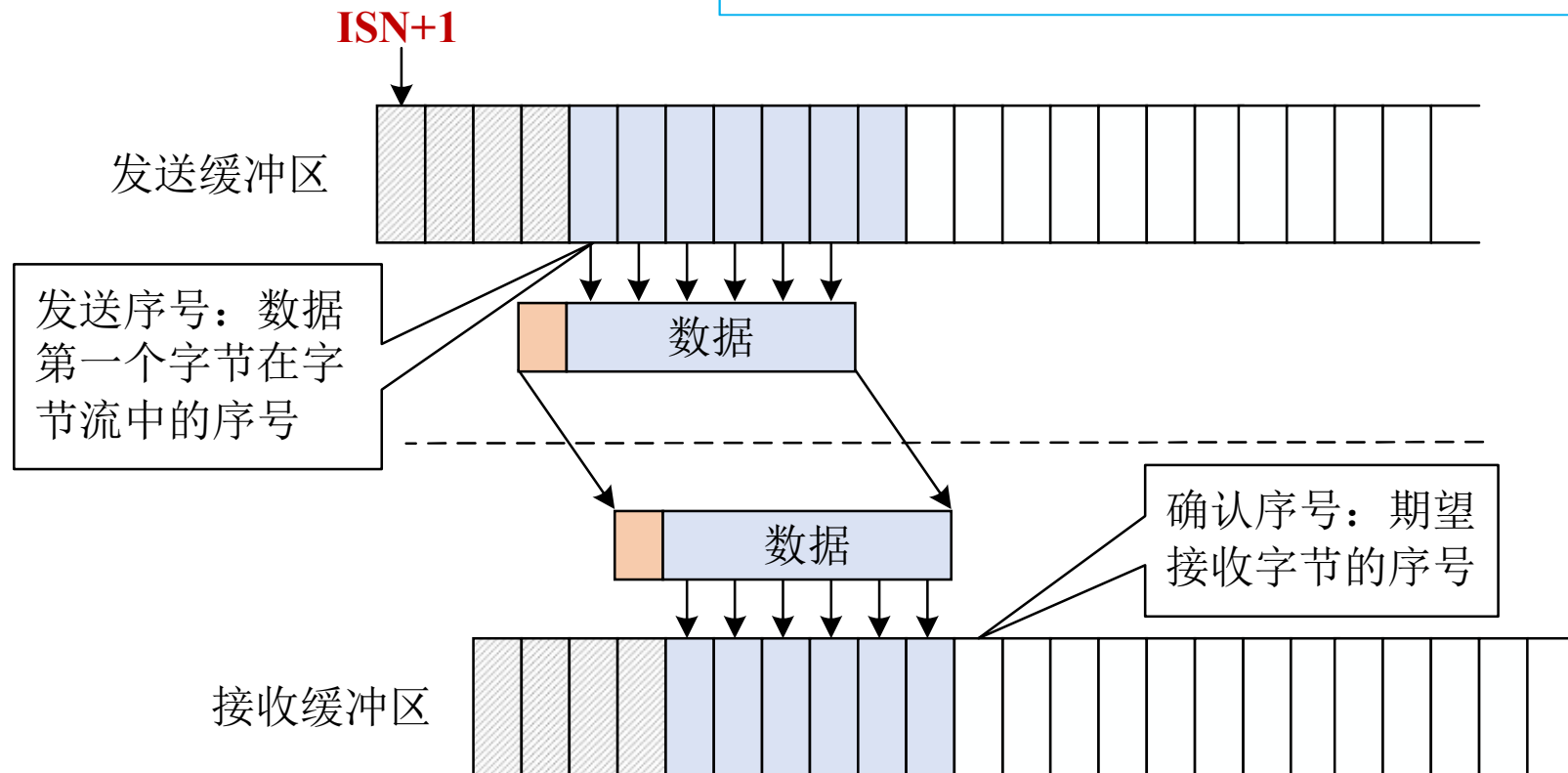
4.2 传输控制协议



4.2.3 TCP可靠数据传输

■ 发送序号与确认序号

TCP是面向字节流的，每个字节数据均有序号，这为分割组装TCP段提供了灵活性，也便于在接收方按序合成字节流



注： TCP支持双向数据流，这里只考虑数据单向流动的场景

4.2.3 TCP可靠数据传输

■ 累积确认

- 确认序号为按序正确接收到的最后一个字节的**下一个序号**，即后继期望接收的字节的序号，指明确认序号之前的数据都已经按序正确接收
- **推迟确认**：接收到TCP段时并不一定立即发送ACK，可能会推迟ACK的发送，但推迟时间必须小于500毫秒，推迟确认主要考虑
 - 应用进程通常是交互的，TCP希望能够通过反向的数据传输将ACK捎带回去，以减少网络带宽消耗
 - 由于TCP是累积确认，对于持续的数据流可以考虑每两个TCP段确认一次，从而减少ACK段的数量

4.2 传输控制协议



4.2.3 TCP可靠数据传输

■ 差错检测

- 利用首部中的校验和对TCP段进行差错检测，校验和的计算方法与UDP类似，但伪首部中的协议字段为6，指明协议类型为TCP
- 如果接收方接收到的TCP段有错误，则丢弃

■ 失序处理

- 接收方对未按顺序到达的TCP段，可以缓存也可以丢弃。目前的TCP实现大都会缓存失序的TCP段，以减少重传段的数量
- 接收到失序的TCP段，**会立即返回ACK段（不推迟确认）**，确认序号为按序正确接收到的最后一个字节的下一个序号

注：接收方是否缓存失序的TCP段，对发送方的处理逻辑没有影响

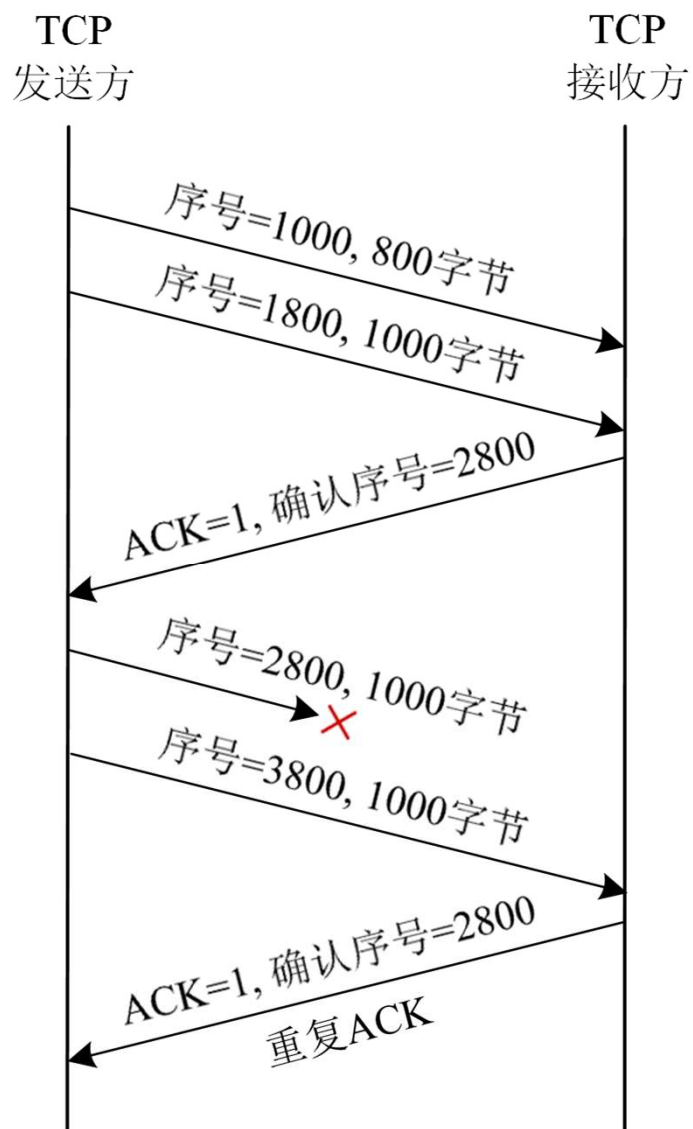
4.2 传输控制协议



4.2.3 TCP可靠数据传输

■ 累积确认和重复ACK示例

- 第一个ACK的确认号为2800，表明接收方按序收到了2800之前的所有字节，期望接收第2800字节
- 序号为3800的数据段正确到达接收方，而序号为2800的数据段未按序到达，接收方不直接对序号为3800的数据段进行确认，而发送重复ACK，告知发送方期望接收第2800字节



4.2 传输控制协议



4.2.3 TCP可靠数据传输

■ 选择确认

- 累积确认的补充，以避免网络资源浪费，提高TCP的性能
- **SACK允许选项**：由发送方在SYN段中携带，在TCP连接建立时向接收方通告发送方接收和处理SACK选项的能力
- **SACK选项**：由接收方在ACK段中携带，对按序接收的数据进行累积确认，对失序的数据块进行选择确认
 - 块左边界为数据块第一个字节的序号，块右边界为该数据块最后一个字节的**下一个序号**。一个数据块需要8个字节，选项长度为 $N*8+2$ 字节，其中N为非连续数据块的个数
 - TCP选项最长为40字节，最多可以包含**4个非连续的数据块**

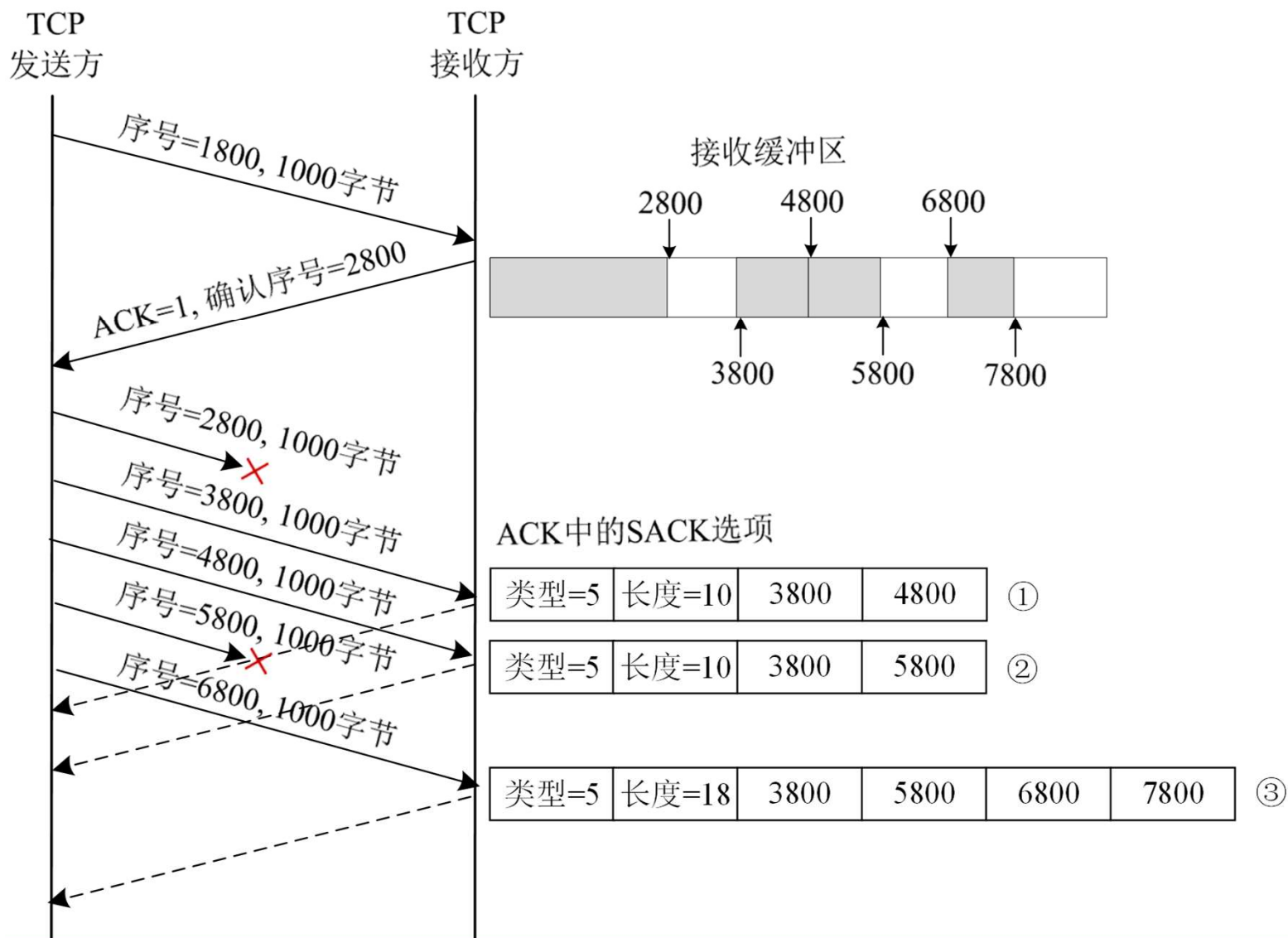
类型=5	长度= $N*8+2$	第1块左边界	第1块右边界	……	第N块左边界	第N块右边界
------	-------------	--------	--------	----	--------	--------

4.2 传输控制协议



4.2.3 TCP可靠数据传输

选择确认示例



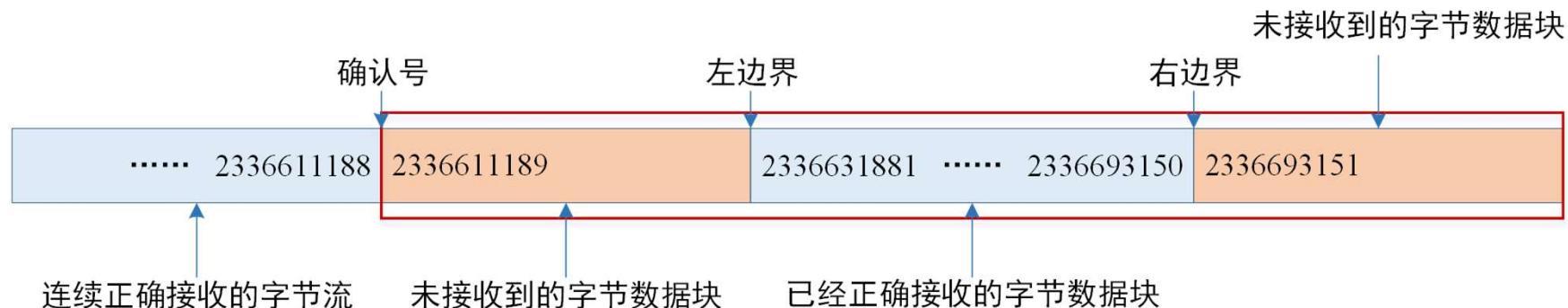
4.2 传输控制协议



4.2.3 TCP可靠数据传输

选择确认示例

```
Transmission Control Protocol, Src Port: 64233, Dst Port: 443, Seq: 1860549586, Ack: 2336611189, Len: 0
Source Port: 64233
Destination Port: 443
[Stream index: 10]
[TCP Segment Len: 0]
Sequence number: 1860549586
Acknowledgment number: 2336611189
1000 .... = Header Length: 32 bytes (8)
Flags: 0x010 (ACK)
Window size value: 65520
[Calculated window size: 65520]
[Window size scaling factor: -1 (unknown)]
Checksum: 0x8beb [correct]
[Checksum Status: Good]
[Calculated Checksum: 0x8beb]
Urgent pointer: 0
Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), SACK
  TCP Option - No-Operation (NOP)
  TCP Option - No-Operation (NOP)
  TCP Option - SACK 2336631881-2336693151
    Kind: SACK (5)
    Length: 10
    left edge = 2336631881
    right edge = 2336693151
    [TCP SACK Count: 1]
```



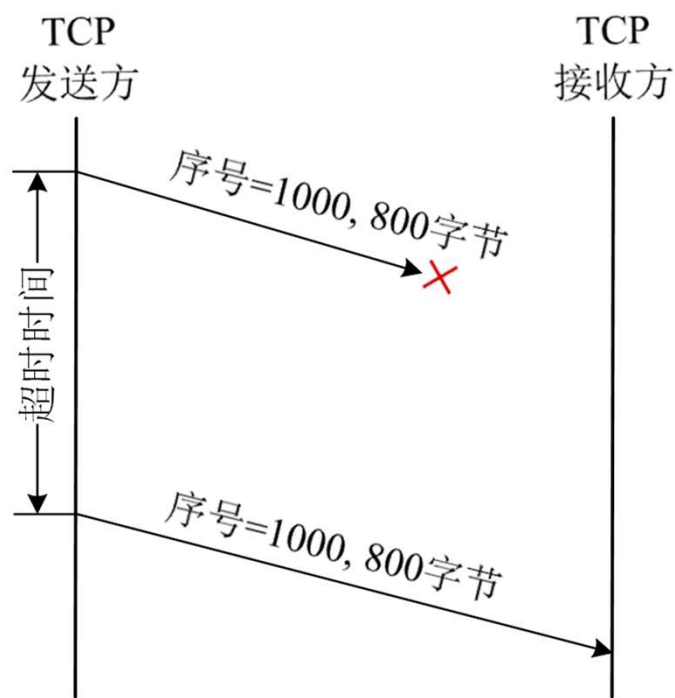
4.2 传输控制协议



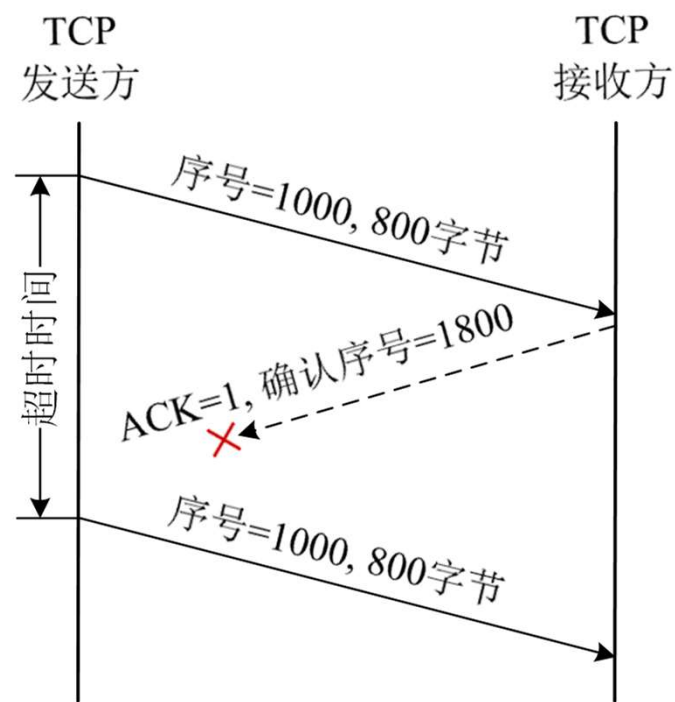
4.2.3 TCP可靠数据传输

■ 超时重传机制

- TCP实体在发出数据段时，启动**重传定时器**。重传定时器超时，仍未接收到接收方返回的ACK，则重传该数据段



(a) 丢失数据段



(b) 丢失ACK

4.2.3 TCP可靠数据传输

■ 重传超时时间（RTO）设定

➤ RTO设定的难点

- TCP是面向互联网的端端传输协议，不同TCP连接的往返时间（RTT）有较大的差别
- 同一个TCP连接，由于网络负载的动态变化及路径的改变，RTT也会发生较大的变化

➤ 不合理的RTO设定对TCP的性能会产生影响

- RTO设置过短，可能会提前超时，引入不必要的重传，浪费网络带宽资源
- RTO设置过长，对于丢失的数据段，等待重传的时间过长，会引入较大的时延

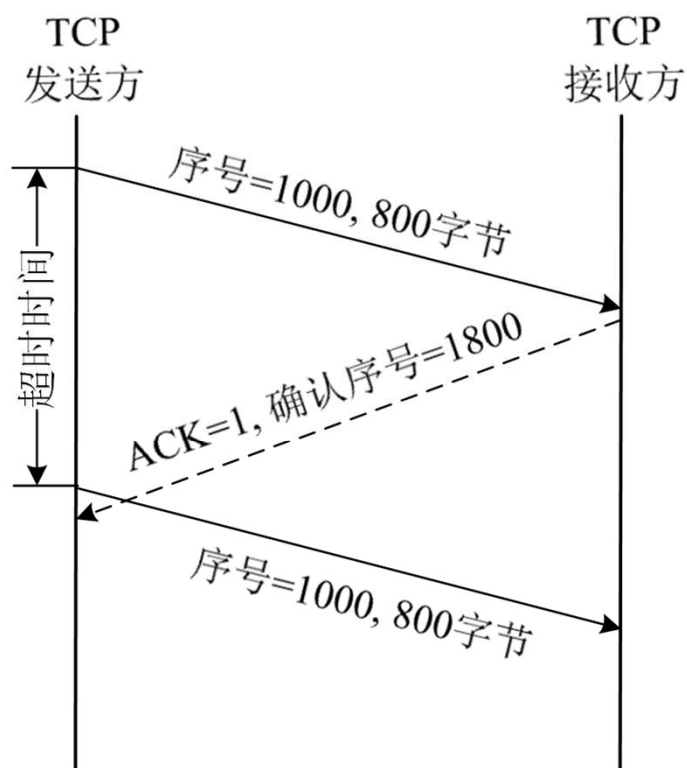
4.2 传输控制协议



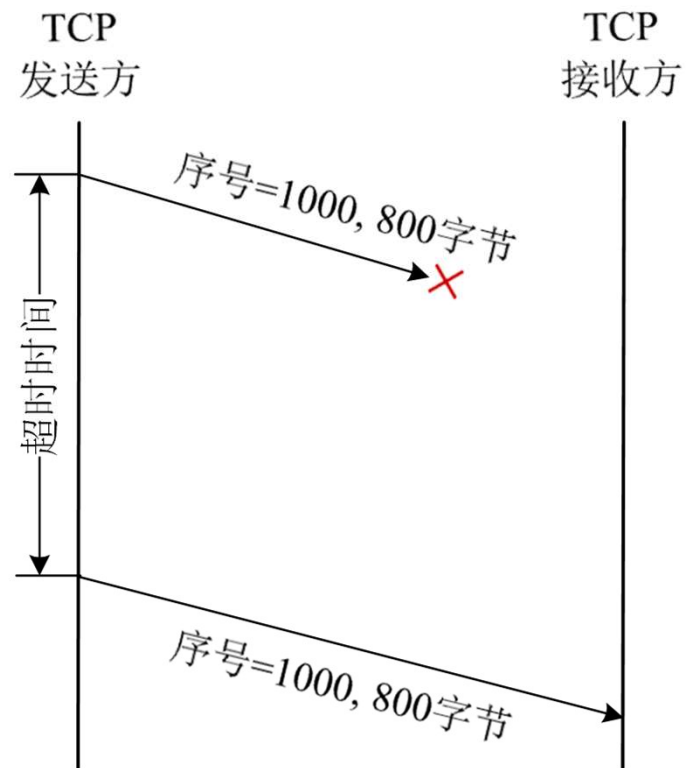
4.2.3 TCP可靠数据传输

■ 重传超时时间（RTO）设定

➤ 过长或过短的超时时间示例



(a) 超时时间过短



(b) 超时时间过长

4.2 传输控制协议



4.2.3 TCP可靠数据传输

■ 重传超时时间（RTO）设定

- TCP采用自适应的RTO计算方法，基于RTT的测量值，不断进行调整，以适应RTT的变化
- **基本RTO估算算法**：对一个TCP连接上的RTT测量值进行**指数加权移动平均**，得到平滑过的往返时间（SRTT）。每次得到新的RTT测量值，SRTT被更新

$$SRTT = \alpha \times SRTT + (1 - \alpha) \times RTT \quad \left(\alpha \text{一般取} \frac{7}{8}\right)$$

$$RTO = \beta \times SRTT \quad (\beta \text{一般取} 2)$$

- 该算法是一种老化算法，新的RTT测量值的权重较大，老的RTT测量值的权重较小，使SRTT能更好地反映网络的当前状况

4.2 传输控制协议



4.2.3 TCP可靠数据传输

■ 重传超时时间（RTO）设定

➤ 示例：假设SRTT的初始值为10， $\alpha = 0.9$ ，测量的RTT序列为10, 6, 14, 6, 14, 10, 8, 25, 26, 22, 6, 10， $RTO = \beta \times SRTT$ ($\beta = 2$)，计算SRTT序列和RTO序列

- RTT: 10, 6, 14, 6, 14, 10, 8, 25, 26, 22, 6, 10
- SRTT: 10, 9.6, 10, 9.6, 10, 10, 9.8, 11.3, 12.8, 13.7, 12.9, 12.6
- RTO: 20, 19.2, 20, 19.2, 20, 20, 19.6, 22.6, 25.6, 27.4, 25.8, 25.2

存在的问题？

4.2.3 TCP可靠数据传输

■ 重传超时时间（RTO）设定

➤ 基本RTO估算算法存在的问题：

- 常数因子灵活性差，当RTT变化范围很大时，上述RTO的计算方法不能很好地适应RTT的变化，可能会引起不必要的重传
- 在网络处于拥塞状态时，不必要的重传会加重网络的负担

➤ Jacobson改进算法：

- 计算RTO时考虑RTT测量值的变化，用RTT均值偏差（DevRTT）反应RTT的变化情况

$$\text{DevRTT} = \delta \times \text{DevRTT} + (1 - \delta) \times |\text{SRTT} - \text{RTT}| \quad (\delta \text{推荐取值 } 3/4)$$

$$\text{RTO} = \mu \times \text{SRTT} + \varphi \times \text{DevRTT} \quad (\mu \text{一般取 } 1, \varphi \text{一般取 } 4)$$

4.2.3 TCP可靠数据传输

■ 重传超时时间（RTO）设定

➤ RTT测量方法

- 在测量RTT时，TCP记录发送数据段的时刻和确认的返回时刻，之间的差值便得到测量的RTT
- 对于在这期间发送的其他数据段，TCP不进行RTT测量，一个RTT间隔内只进行一次测量

➤ RTT测量问题

- 如果用于RTT测量的数据段由于超时被重传，发送方无法判断接收到的ACK是针对原始数据段的确认，还是针对重传数据段的确认，存在二义性，导致TCP无法准确测量RTT

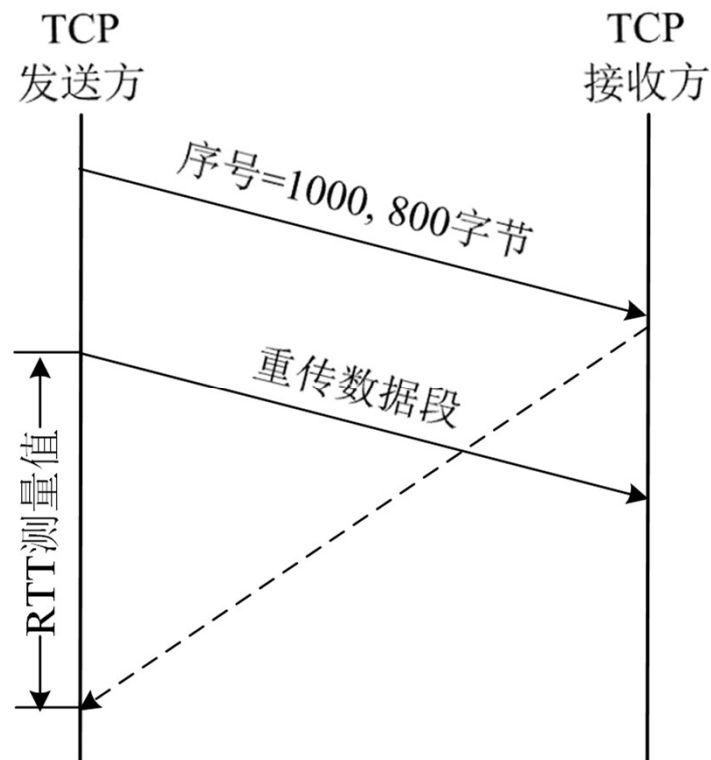
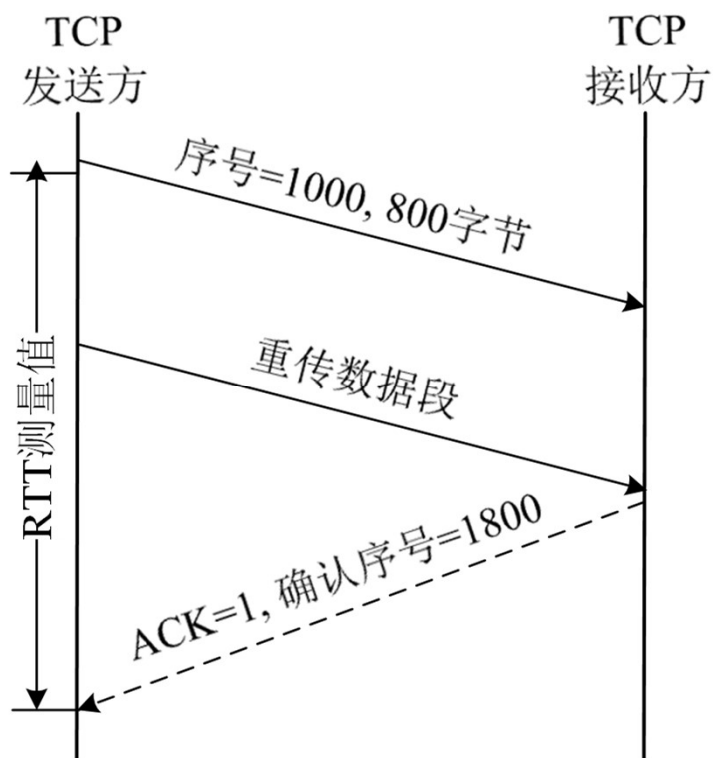
4.2 传输控制协议



4.2.3 TCP可靠数据传输

■ 重传超时时间（RTO）设定

➤ RTT测量问题示例



4.2 传输控制协议



4.2.3 TCP可靠数据传输

■ 重传超时时间（RTO）设定

➤ Karn算法

- 用于测量RTT的数据段如果被重传，则停止RTT的测量，不再重新计算SRTT和RTO，将RTO设置成上一次的2倍
- 重传数据段被确认后，重新启动RTT测量
- **问题**：重传数据段较多时，不能有效测量RTT

➤ 利用时间戳选项测量RTT

- 发送方发送数据段时刻记为**TS1**，该值被放入“时间戳值”字段
- 接收方在返回ACK时，把数据段中的“时间戳值”复制到“时间戳回送”字段
- 发送方接收到ACK的时刻记为**TS2**，则 **$RTT=TS2-TS1$**

类型=8	长度=10	时间戳值	时间戳回送
------	-------	------	-------

4.2.3 TCP可靠数据传输

■ 快速重传机制

- **超时重传的问题**：RTO相对较长，超时重传会引入较长的时延
- **解决策略**：通过3次重复ACK检测TCP段丢失，在超时之前重传丢失的TCP段，以降低时延，称之为**快速重传**
 - 发送方连续发出多个数据段，如果一个（或几个）数据段丢失，而其后续的数据段正确到达，接收方立即返回ACK，发送方会接收到多个重复的ACK
 - 如果发送方接收到对同一个序号3次重复的ACK，表明该序号的数据段之后已经有3个数据段到达了接收方。发送方可以据此推断该序号的数据段丢失，并立即重传丢失的数据段

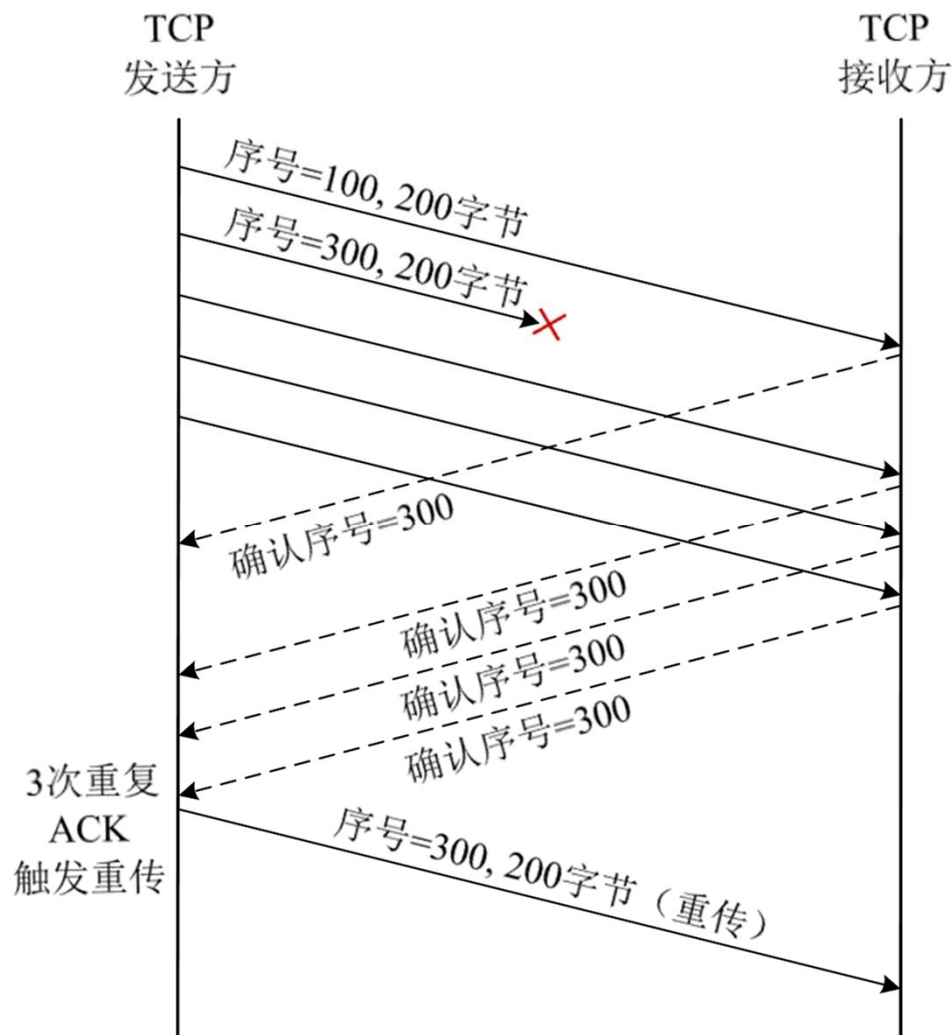
4.2 传输控制协议



4.2.3 TCP可靠数据传输

■ 快速重传示例

- 序号为100的数据段被正确收到，接收方返回确认序号为300的ACK段
- 序号为300的数据段丢失，之后的3个数据段到达接收方
- 接收方返回3个确认序号为300的重复ACK
- 发送方收到3个重复ACK，重传序号为300的数据段



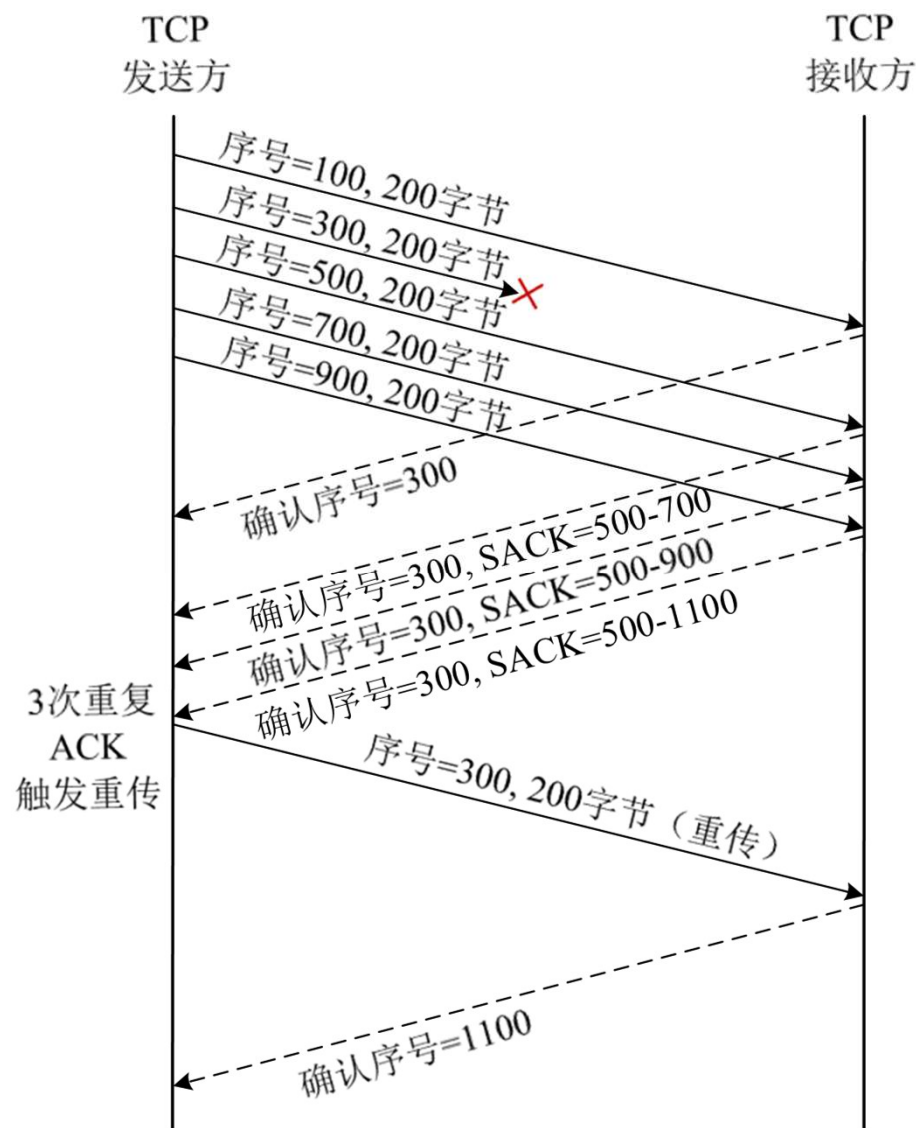
4.2 传输控制协议



4.2.3 TCP可靠数据传输

■ 快速重传与SACK结合

- 接收方缓存失序的数据段，在SACK选项中进行确认
- 接收方接收到序号为300的重传数据段时，序号为1100之前的数据都被按序正确接收，返回确认序号为1100的ACK



4.2.3 TCP可靠数据传输

■ 流量控制

- **发送窗口**：允许发送方发出未确认的最大数据量
 - 受发送缓存能力约束：发出未确认的数据均需要缓存，以备重传
 - 受接收方接收能力约束（**流量控制**）
 - 受网络传输能力约束（**拥塞控制**）
- **流量控制的目标**：使发送方尽可能快地发送数据，但不能造成接收缓冲区溢出，而导致TCP段被丢弃
- **基于窗口的流量控制**：
 - 发送方维护一个发送窗口，接收方维护一个接收窗口
 - 接收窗口大小通过ACK段中的“接收窗口通告”告知发送方，发送方根据该通告调整发送窗口，初始接收窗口大小在连接建立时通告

4.2 传输控制协议

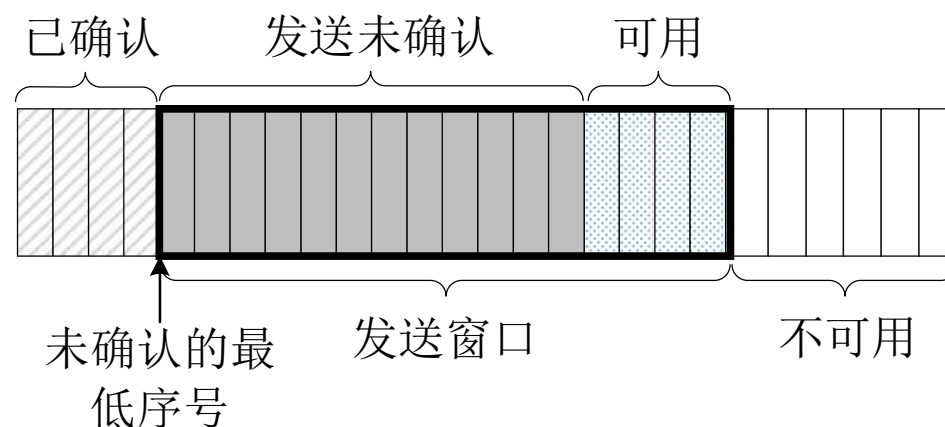


4.2.3 TCP可靠数据传输

假设序号是无限的，32位序号循环使用

■ 流量控制：发送窗口

- 发送窗口的左边界是未确认的最低序号，之前的数据都已被确认
- 发送的未确认数据用完发送窗口中的全部序号，停止发送
- 接收到ACK段，其确认序号大于左边界的序号时，发送窗口的**左边界**向右移动到确认序号指示的位置
- 通告窗口大于当前发送窗口时，发送窗口的**右边界**向右移动
- 如果ACK段中包含SACK选项，其所确认的序号不会引起发送窗口左边界移动，只需要把确认的序号标记为已确认



4.2 传输控制协议

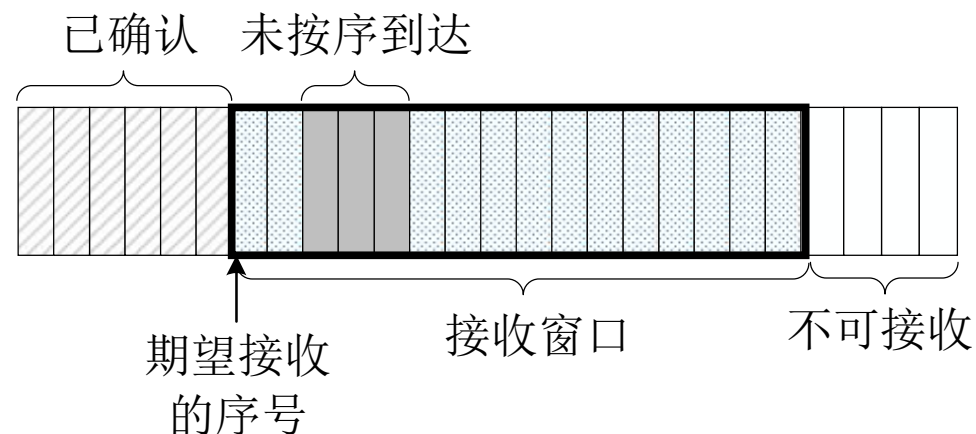


4.2.3 TCP可靠数据传输

假设序号是无限的，32位序号循环使用

■ 流量控制：接收窗口

- 接收窗口的左边界是下一个期望接收的序号，它之前的数据都已按序正确收
- 当接收到按序到达的数据时，接收窗口的左边界向右移动
- 当接收方的应用进程读取数据后，该部分数据所占用的接收缓冲区空间被释放，接收窗口的右边界向右扩展
- 落入接收窗口中未按序到达的数据，接收方可以进行缓存，但接收窗口不发生变化。接收方可以随ACK段发送SACK选项，对失序数据块进行确认

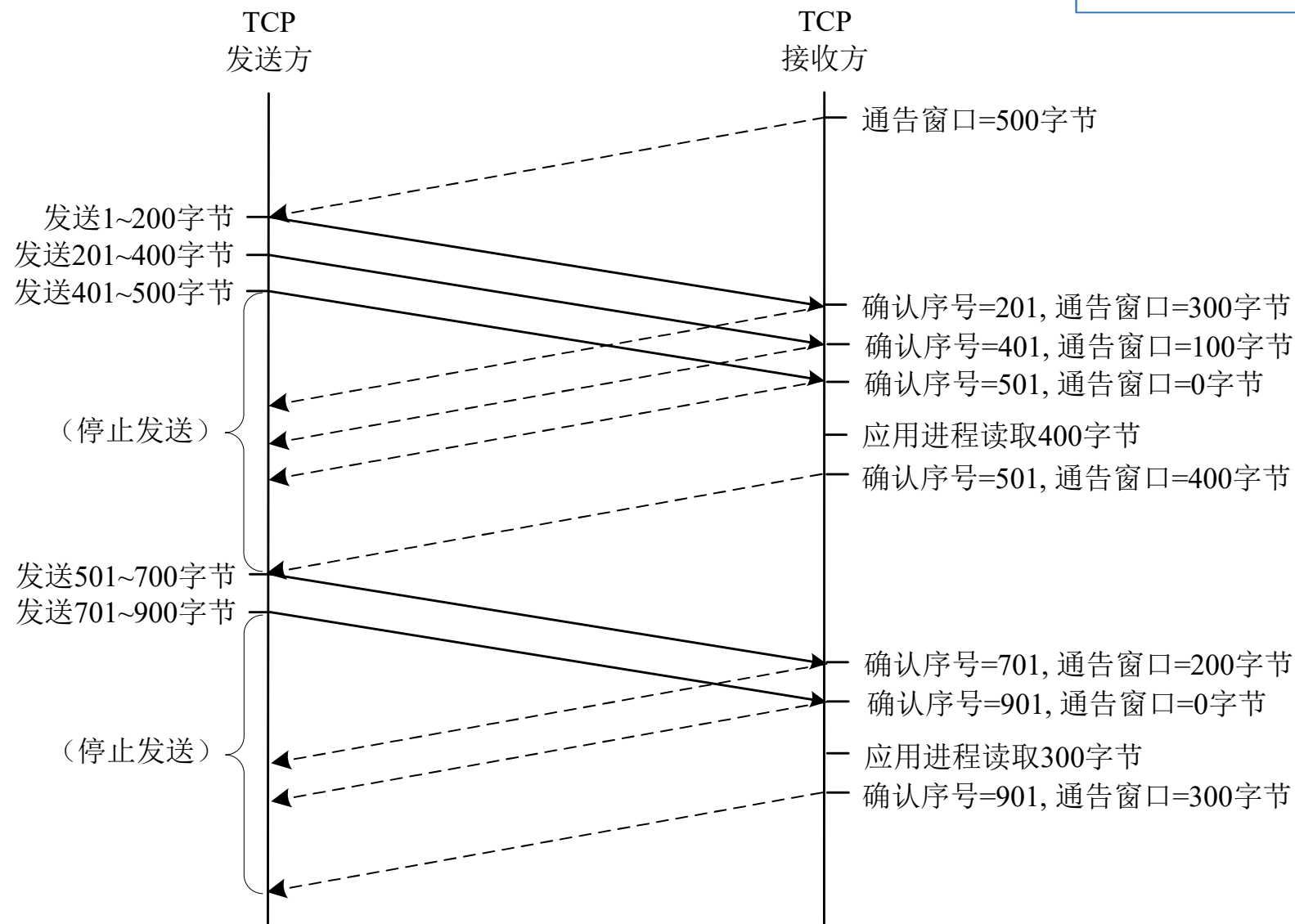


4.2 传输控制协议



4.2.3 TCP可靠数据传输

流量控制示例



4.2 传输控制协议



4.2.3 TCP可靠数据传输

注意： TCP不保证ACK段的可靠传输

■ 流量控制：窗口探测

- **问题：** 发送方接收到窗口为零通告，后面接收方发送的非零窗口通告丢失，发送方如何获知接收窗口不再为零？
- **解决策略：**
 - 发送方在接收到零窗口通告且有数据待发送时，启动一个定时器，定时器初始值设成RTO
 - 定时器超时，发送方发送一个只包含1个字节数据的**窗口探测**段，同时重新启动定时器
 - 接收方接收到窗口探测段后返回ACK，其中通告当前接收窗口的大小
 - 如果接收方的当前接收窗口仍为零，则丢弃窗口探测段中的数据
 - 发送方持续发送窗口探测段，直至获得非零窗口通告

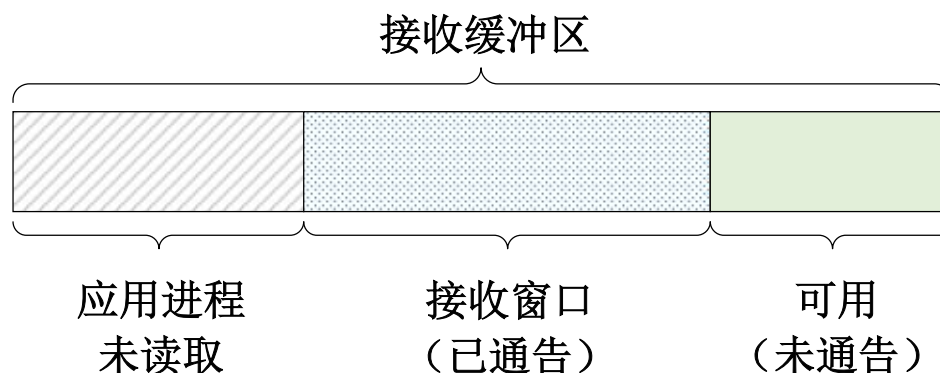
4.2 传输控制协议



4.2.3 TCP可靠数据传输

■ 流量控制：糊涂窗口综合症（SWS）

- **问题**：接收方在接收缓冲区有少量可用空间时立即向发送方进行窗口通告，发送方会发送小数据段，小数据段会产生相对大的额外开销，大量的小数据段会降低网络资源的利用率
- **接收方解决策略**：限制发送小增量的窗口更新通告
 - 当**可以使用但还未通告部分**大于接收缓冲区的一半或达到一个MSS时，才移动接收窗口的右边界，并向发送方通告窗口的变化



4.2 传输控制协议



4.2.3 TCP可靠数据传输

■ 流量控制：糊涂窗口综合症（SWS）

- **发送方解决策略**：尽可能减少小数据段（长度小于MSS）的发送
- **Nagle算法**：确定何时能够发送一个小数据段
 - 发送窗口中有足够的可用序号且应用进程写入了足够的数据，发送方可以封装一个满载数据段（包含MSS字节数据），立即发送
 - 在发送窗口中所有数据都被接收方确认后，可以发送一个小数据段
- **Nagle算法的优点**：
 - 具有自适应性，确认到达的越快，数据也发送的越快，可以有效降低低速广域链路上小数据段的数量
- **Nagle算法的问题**：
 - 会引入较大的延迟，不适合对时延要求较高的交互式应用

4.2 传输控制协议



4.2.3 TCP可靠数据传输

■ 流量控制：接收窗口扩展

➤ **问题：**16位接收窗口通告在有些情况下会影响TCP连接的吞吐率

- 例如，一个TCP连接的RTT为20毫秒，带宽为1Gb/s，接收窗口为65535字节，发送方发送65535字节数据所用的传输时间为

$$\text{传输时间} = \frac{65535 \times 8}{1 \times 10^9} \approx 0.5 \text{ 毫秒}$$

- TCP连接的吞吐率约为

$$\text{吞吐率} = \frac{\text{传输时间}}{\text{RTT}} \times 1\text{Gb/s} = \frac{0.5}{20} \times 1\text{Gb/s} = 25\text{Mb/s}$$

- 对于**长肥管道**（高带宽、长往返时间的TCP连接）这种问题更为突出

4.2 传输控制协议



4.2.3 TCP可靠数据传输

■ 流量控制：接收窗口扩展

➤ 解决策略：扩大接收通告窗口的大小

- 在前面例子中，将接收窗口扩大10倍，其吞吐率可以达到250Mb/s

➤ 窗口扩展因子选项：对通告的接收窗口进行扩展

- 移位数取值在0~14之间，0代表没有扩展
- 移位数用 S 表示，通告窗口为 W ，接收窗口可以扩展到 $W \times 2^S$
- 接收窗口最大可以扩展到 65535×2^{14} ，约为 2^{30} 。TCP序号空间为 2^{32} ，大于 2×2^{30} ，满足滑动窗口的基本要求
- 窗口扩展因子选项只能在SYN段中出现，连接一旦建立，扩展因子选项是固定的

类型=3	长度=3	移位数（1字节）
------	------	----------

4.2.4 TCP拥塞控制

- **前情回顾**：互连层采用分组交换方式转发IP数据包，链路共享采用统计时分多路复用方式。受限于路由器的转发能力和链路带宽，当IP数据包到达过快时路由器可能不能及时对数据包进行转发，会在路由器中排队等待
- **网络拥塞**：数据包在网络中经历**较长的排队延时**或由于**缓冲队列溢出而被丢弃**，则出现网络拥塞
 - 在分组交换网络中，为了有效地利用链路带宽资源，路由器中的适当排队是可以接受的，但长时间排队及数据包丢失应该避免
- **拥塞控制的目标**：
 - 尽可能快地传输数据，但要避免网络拥塞发生
 - 在网络拥塞发生时能够进行有效恢复

4.2.4 TCP拥塞控制

■ 关键问题

- 如何感知网络的状态
- 如何控制发送速率

■ 网络状态感知

- 网络辅助的方法：
 - 路由器提供显式反馈
 - 例如，TCP的显示拥塞通知机制（ECN）
- 基于主机的方法：
 - 路由器不提供明确的反馈，由主机根据数据流的某些特征进行推断
 - 主机通过观察数据的丢失、延迟等情况推断是否发生拥塞及发生拥塞的程度

4.2.4 TCP拥塞控制

■ 发送速率控制

- TCP主要采用**基于拥塞窗口**的拥塞控制方法
- 利用拥塞窗口的增减控制发送方能够发出未确认数据的数量
- **关键问题**：如何确定拥塞窗口的大小。过大的拥塞窗口可能会造成网络拥塞，较小的拥塞窗口会降低网络资源的利用率及TCP连接的吞吐率

■ 典型的TCP拥塞控制算法

- TCP拥塞控制算法种类很多，代表性的包括Reno与New Reno算法、CUBIC、BBR、DCTCP等等

注：TCP发送窗口的实际大小取决于**接收方通告窗口**和**拥塞窗口**的小者，即发送方的发送速率受网络的转发能力和接收方的接收能力共同约束

4.2 传输控制协议



4.2.4 TCP拥塞控制：TCP Reno

- Reno算法是传统的TCP拥塞控制算法，最早在UNIX系统中实现，后被各种操作系统广泛支持
- Reno算法以数据段丢失作为网络拥塞的标志，通过调整拥塞窗口的大小控制发送方的发送速率
 - 前提假设：由于传输错误造成数据段被丢弃的几率很小（远小于1%），数据段丢失主要源于网络拥塞
- Reno算法包括慢启动、拥塞避免、快速恢复三种机制
 - 拥塞窗口(*cwnd*)和慢启动阈值(*ssthresh*)：以MSS作为计数单位
 - 当*cwnd*小于*ssthresh*时，处于慢启动状态
 - 当*cwnd*达到*ssthresh*时，进入拥塞避免状态

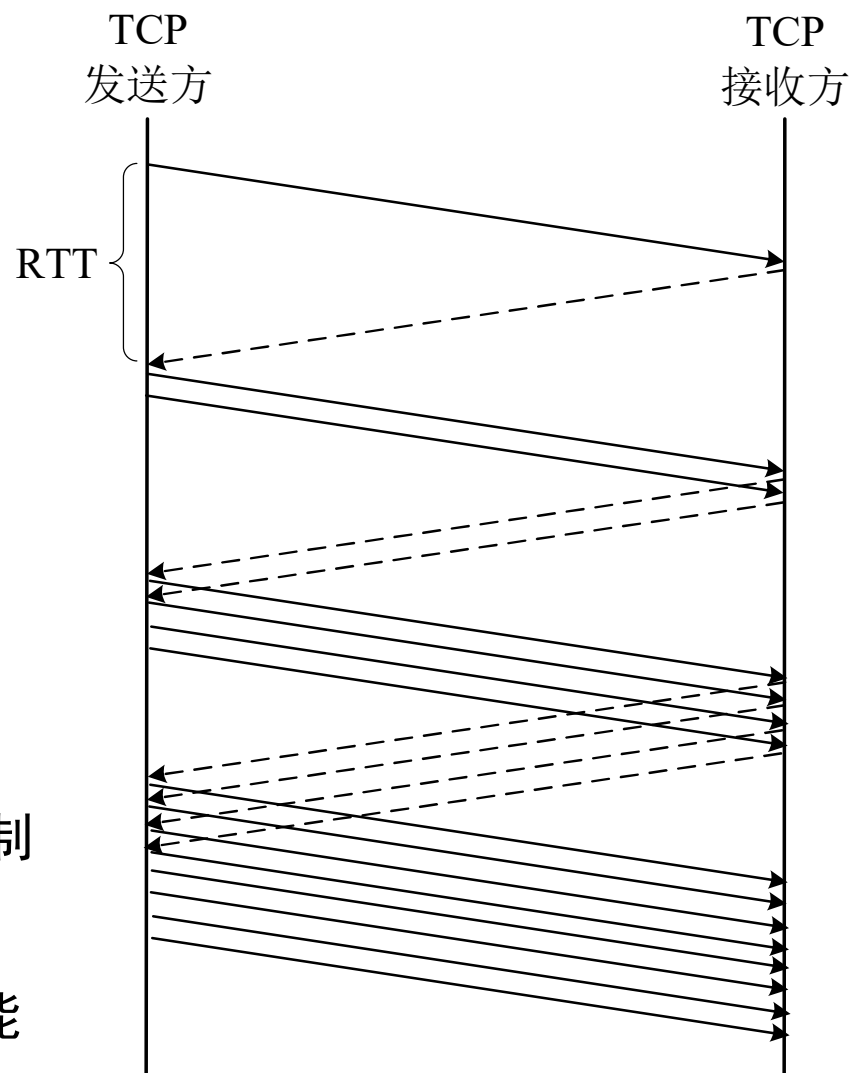
4.2 传输控制协议



4.2.4 TCP拥塞控制：TCP Reno

■ 慢启动机制

- 连接建立之初或重传定时器超时未得到确认时，进入慢启动状态
 - 初始cwnd 设置为1
 - 每收到1个新ACK, *cwnd* 增1, 即每个RTT, *cwnd* 翻倍
- 拥塞窗口呈指数增长, 使发送速率能够快速接近网络的可用带宽
- 对于长肥管道, 初始 *cwnd* = 1 会限制TCP连接的吞吐率, 可以设置较大的 *cwnd* 初始值, 以提高TCP的性能



4.2 传输控制协议

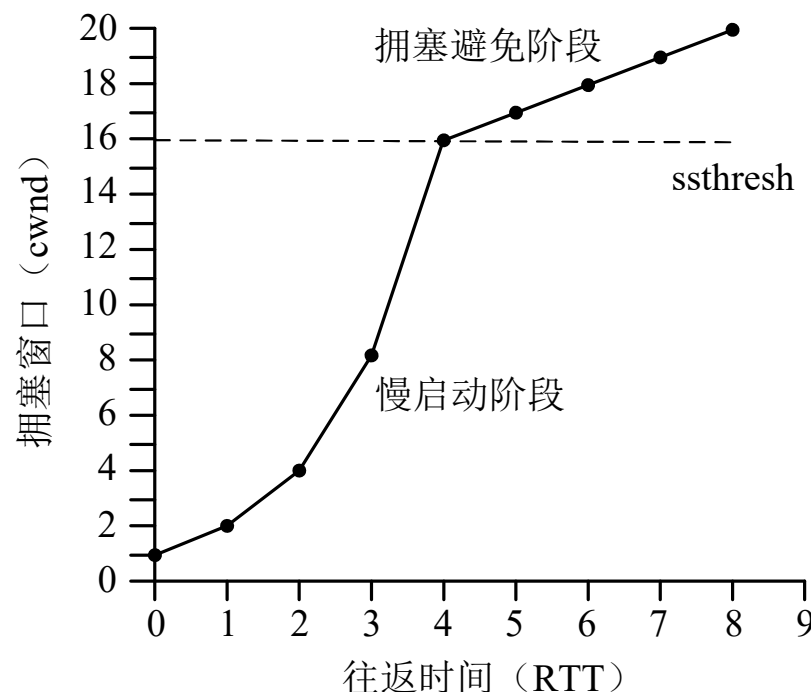


4.2.4 TCP拥塞控制：TCP Reno

■ 拥塞避免机制

- 当 *cwnd* 达到 *ssthresh* 时，进入拥塞避免状态
 - 拥塞窗口中的所有数据段都得到确认，*cwnd* 增1
 - 理论上可以简单视为经过1个RTT，*cwnd* 增1
- 拥塞窗口呈线性增长，相对慢启动状态增长速度减缓，尽可能避免拥塞发生
- 发送方通常在每收到一个ACK，*cwnd* 按如下方式增长

$$cwnd = cwnd + \frac{1}{cwnd}$$



4.2 传输控制协议



4.2.4 TCP拥塞控制：TCP Reno

■ 拥塞感知

➤ Reno算法通过数据段的丢失感知网络拥塞

➤ 重传定时器超时检测丢失：

- $ssthresh = cwnd / 2$
- $cwnd = 1$ （进入慢启动状态）

➤ 三次重复ACK检测丢失：

- $ssthresh = cwnd / 2$
- $cwnd = ssthresh + 3$ （进入快速恢复状态）

注：收到三次重复ACK，可以获知丢失数据段后面至少有3个数据段已经到达接收方，收发双方之间仍然有数据流动，网络拥塞不十分严重

■ 快速恢复机制

➤ 每收到一个重复ACK， $cwnd$ 增1

➤ 收到新的ACK，将 $cwnd$ 设置成 $ssthresh$ ，进入拥塞避免状态

4.2 传输控制协议



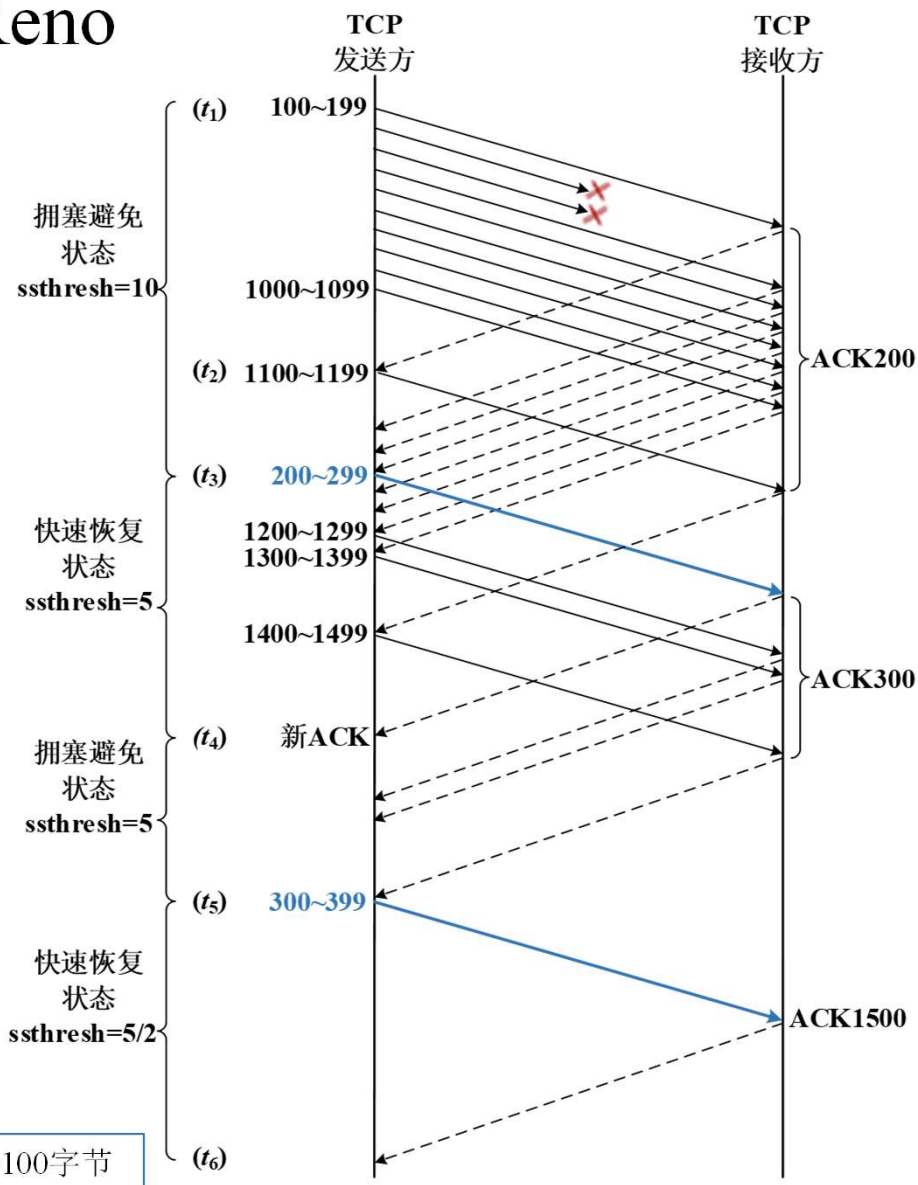
4.2.4 TCP拥塞控制：TCP Reno

■ Reno算法示例

- 假设没有超时事件发生

■ 问题分析

- 一次网络拥塞引起多个数据段丢失，会使拥塞窗口多次折半
- 后继没有足够的数据段发送或不能发送更多的数据段时，只能等到超时才能被重传，并进入慢启动状态，
- **结果：**增大丢失数据段的恢复时间，降低TCP的吞吐率



4.2 传输控制协议



4.2.4 TCP拥塞控制：TCP Reno

■ NewReno算法：主要解决拥塞窗口多次折半问题

- **Reno问题的原因**：过早地退出了快速恢复状态
- **NewReno的两个概念**：部分确认（PACK）和恢复确认（RACK）
 - 进入快速恢复状态时记录发出且未得到确认字节的最大序号 y
 - 在快速恢复阶段接收到新的ACK的确认序号大于 y ，该确认为RACK，否则为PACK
- **NewReno的基本思想**
 - 发送方在快速恢复状态收到PACK时，不会退出快速恢复状态，而是重传下一个丢失的数据段
 - 发送方收到RACK时， $cwnd$ 设置为 $ssthresh$ ，并退出快速恢复状态
 - NewReno每个RTT重传一个丢失的数据段，在一个发送窗口中有 m 个数据段丢失，快速恢复状态将持续约 m 个RTT

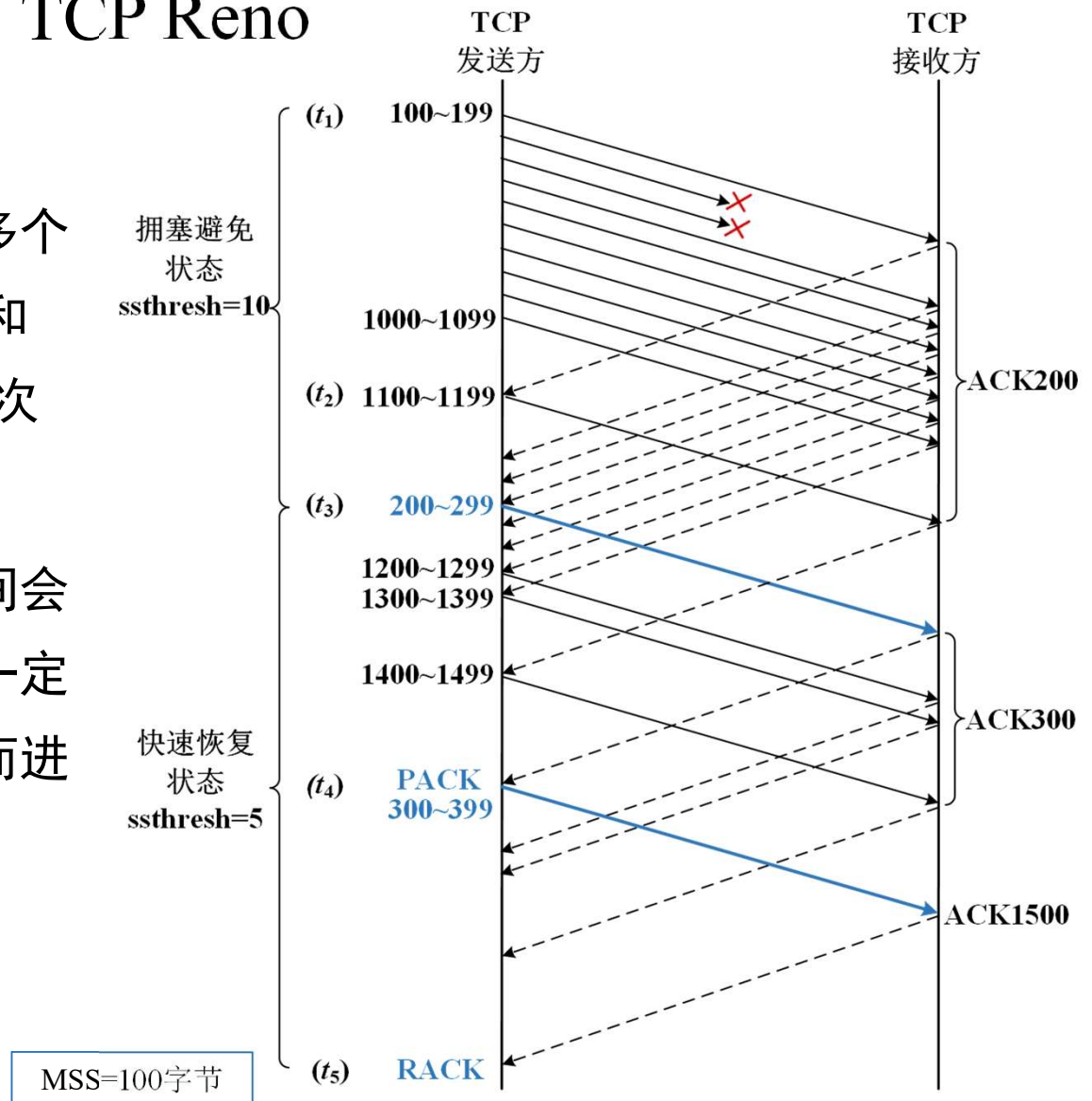
4.2 传输控制协议



4.2.4 TCP拥塞控制：TCP Reno

■ NewReno算法示例

- 在一个拥塞窗口内有多个数据段丢失时，*cwnd*和*ssthresh*的值没有被多次折半
- 丢失数据段的恢复时间会有所降低，也可以在一定程度上避免由于超时而进入慢启动阶段的问题



4.2 传输控制协议



4.2.4 TCP拥塞控制：TCP CUBIC

■ Linux内核2.6之后的默认TCP拥塞控制算法

■ 解决的主要问题

- Reno在拥塞避免阶段拥塞窗口增长缓慢，每个RTT增1，在高速长距离网络中不能充分利用网络带宽资源，会限制TCP的吞吐率
- Reno拥塞窗口的增长依赖于RTT，RTT越小拥塞窗口增速越快，在不同RTT的TCP连接之间存在公平性问题

■ CUBIC算法的特点

- 主要针对拥塞避免阶段进行改进，使用三次函数的凸凹曲线代替线性增长，可以提高在高速长距离网络中的可扩展性和稳定性
- 拥塞窗口的增长独立于RTT，能更好地保证TCP连接之间的公平性

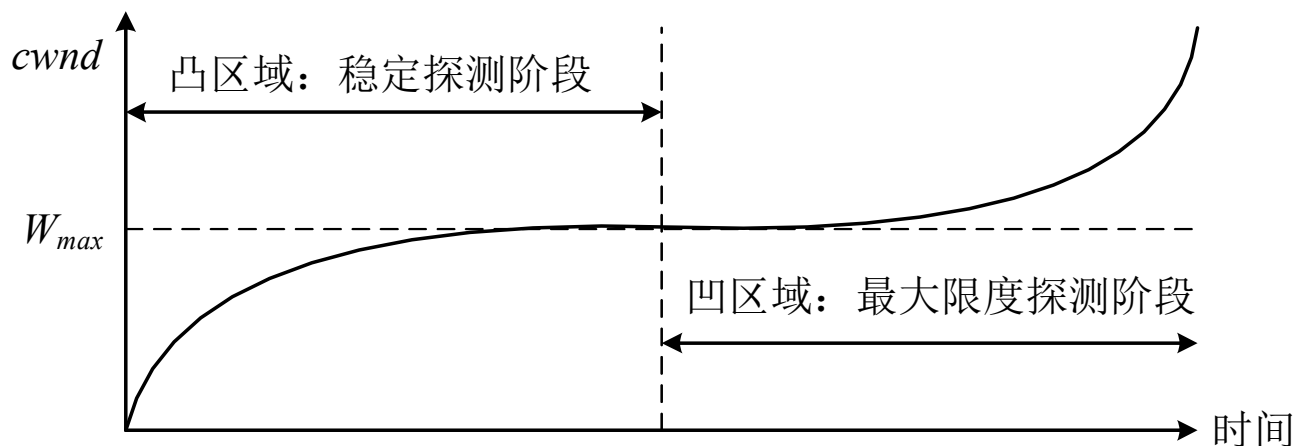
4.2 传输控制协议



4.2.4 TCP拥塞控制：TCP CUBIC

■ CUBIC算法的基本思想

- 当TCP通过三次重复ACK或显示拥塞通知检测到拥塞事件时，则将该时刻拥塞窗口的大小保留在变量 W_{max} 中，并对拥塞窗口和慢启动阈值进行乘性减少，即 $ssthresh = W_{max} \times \beta$ ， $cwnd = W_{max} \times \beta$ ， β 为乘性递减因子，取值为0.7
- 利用三次函数的凸形曲线增加拥塞窗口，三次函数的稳定点设置为 W_{max} 。在窗口大小达到 W_{max} 之后，三次函数进入凹形曲线区域



4.2 传输控制协议



4.2.4 TCP拥塞控制：TCP CUBIC

■ CUBIC算法的基本思想

- CUBIC将拥塞窗口作为自上次拥塞事件以来所经过时间 t 的函数，三次函数曲线为

$$cwnd(t) = C \times (t - K)^3 + W_{max}$$

- C 为常数，决定CUBIC在与其他拥塞控制算法竞争带宽方面的抢占性。在公平性和网络资源利用率方面进行权衡，建议 C 取0.4
- K 是基于上述函数将窗口增加到 W_{max} 所需的时间，如果没有进一步的拥塞事件发生，则 K 用下面公式计算

$$K = \sqrt[3]{\frac{W_{max} \times (1 - \beta)}{C}}$$

4.2 传输控制协议



4.2.4 TCP拥塞控制：TCP CUBIC

■ CUBIC算法进一步说明

- CUBIC在连接建立之初，以及通过超时检测到丢失事件时依然执行慢启动算法
 - 可以使用传统的TCP慢启动算法，也可以针对高速长距离网络选择改进的慢启动算法
- $cwnd$ 达到 $ssthresh$ 时，慢启动结束，进入拥塞避免阶段，使用三次函数曲线公式增大拥塞窗口
 - W_{max} 设置为进入拥塞避免时的 $cwnd$ 值， t 为自当前拥塞避免开始以来经过的时间， K 设置为0，直接进入**最大限度探测阶段**

4.2.4 TCP拥塞控制：BBR算法

■ 基于丢失事件的拥塞控制方法面临的问题

➤ 只有在检测到丢失事件时才会降低拥塞窗口

- **何时会丢包**：如果采用基本的先入先出调度策略，只有在瓶颈链路的缓冲区被填满时才会发生丢包

➤ **缓冲区膨胀**：

- 为了避免丢包，路由器不断增大缓冲区的大小
- 大的缓冲区可以容纳较多的数据，但会形成较长的缓冲队列
- 需要较长的时间才能将缓冲区中的数据排空，会引入过长的排队延时

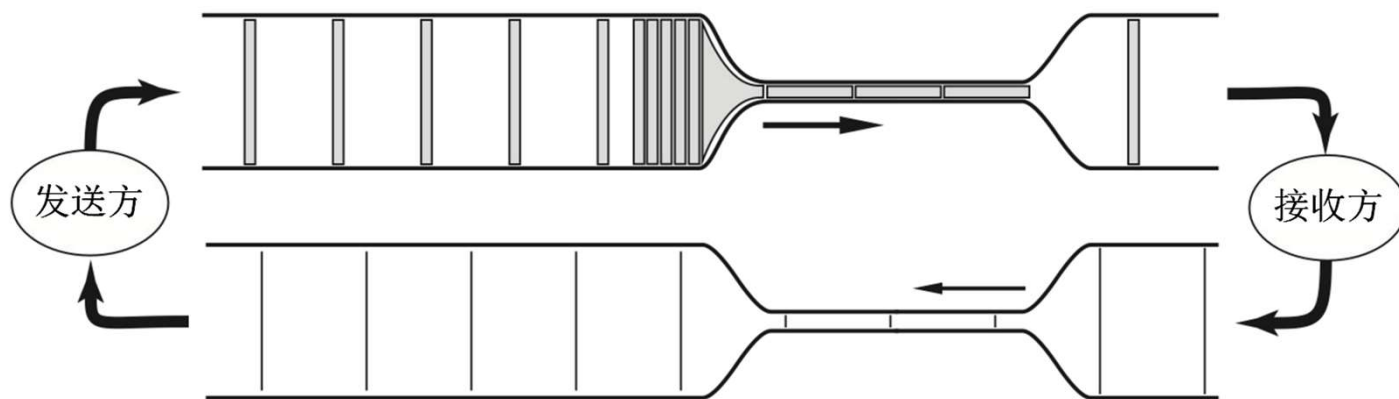
➤ **问题**：缓冲区膨胀问题对交互式应用会产生较大的影响

4.2 传输控制协议



4.2.4 TCP拥塞控制：BBR算法

- BBR (Bottleneck Bandwidth and Round-trip propagation time) 算法是2016年由谷歌公司设计的一种拥塞控制算法，目前演化到第2版，已集成到Linux内核中
- BBR不再基于丢失事件感知网络拥塞，而是**主动探测网络带宽**
 - **瓶颈链路带宽**决定端到端路径上的**最大数据投递速率**
 - 过高的数据发送量会在路由器缓冲区中排队，增加网络延时



4.2 传输控制协议



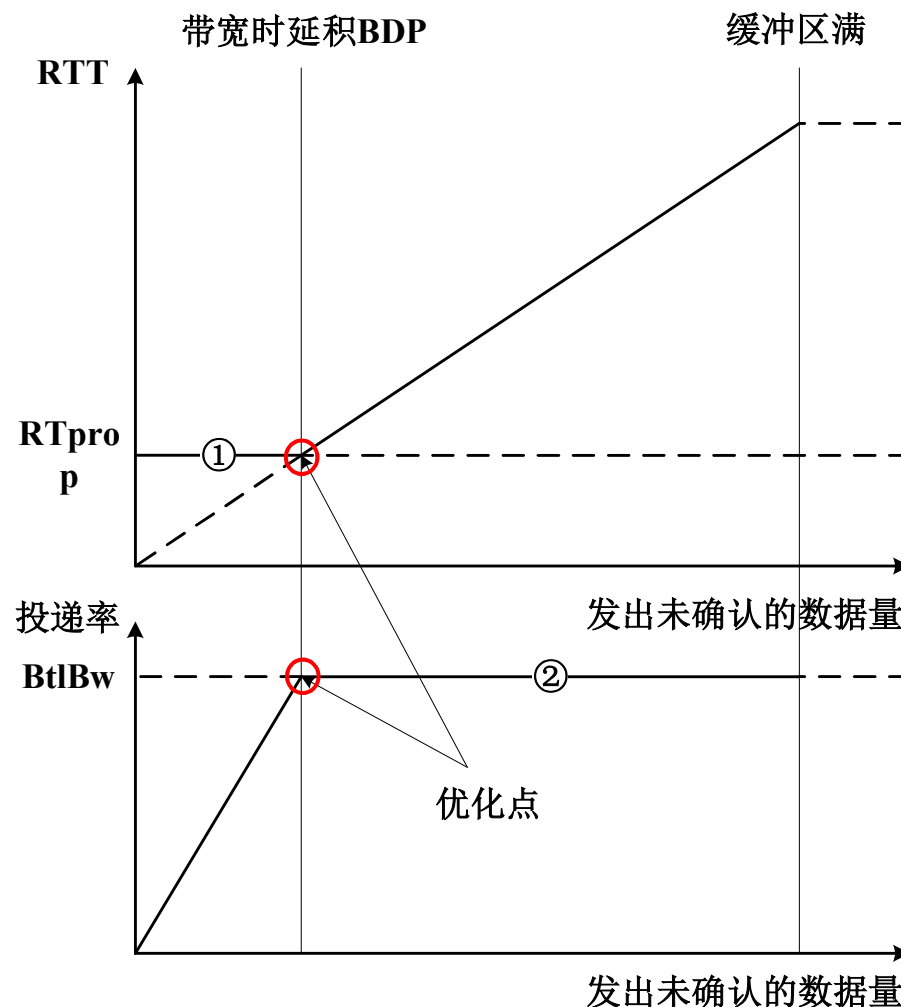
4.2.4 TCP拥塞控制：BBR算法

■ 核心目标

- 确定合适的的数据发送量，以获得**最高投递率**和**最低延时**，从而充分利用网络带宽，降低链路缓冲区的占用率

■ 需要解决的问题

- 测量最大瓶颈带宽**BtlBw**和最小往返传播延时**RTprop**
$$BDP = BtlBw \times RTprop$$
- 基于**BDP**确定拥塞窗口及发送速率，理论上数据不会在路由器的缓冲区中排队



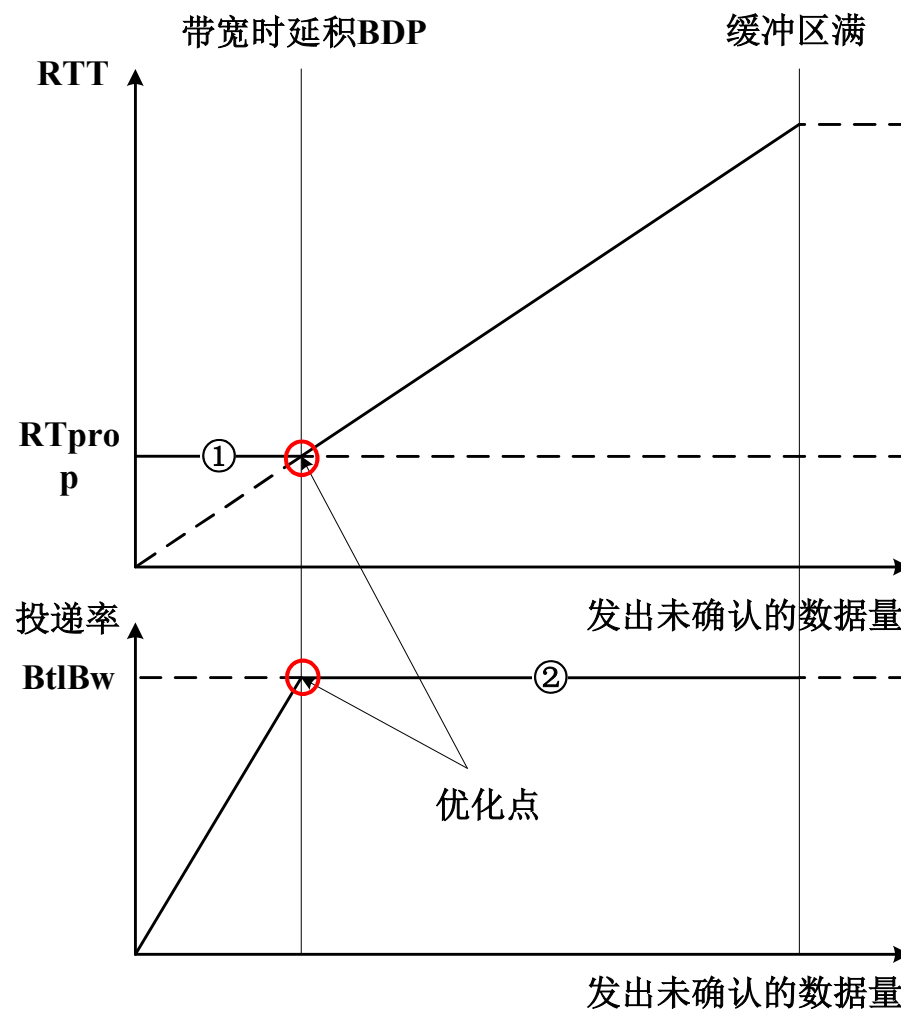
4.2 传输控制协议



4.2.4 TCP拥塞控制：BBR算法

■ 难点： RT_{prop} 和 $BtlBw$ 很难同时被测量

- 要测量 $BtlBw$ ，需要把瓶颈链路填满，此时路由器的缓冲区中存在排队，延迟较高
- 要测量 RT_{prop} ，需要保证路由器的缓冲区不被占用
- BBR采用优化点的近似观测，即用过去10秒内的最小RTT和最大投递率，分别作为近似的 RT_{prop} 和 $BtlBw$ ，并依据这两个值估算当前的 BDP



4.2 传输控制协议



4.2.4 TCP拥塞控制：BBR算法

■ BBR根据 $BtlBw$ 和 BDP 设定拥塞窗口和发送速率

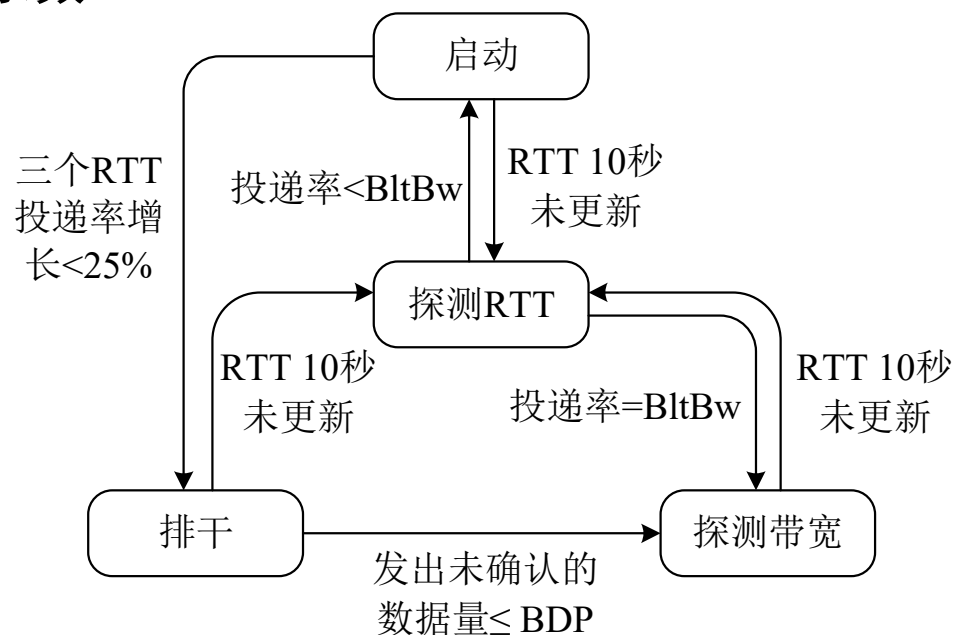
- 拥塞窗口: $cwnd = cwnd_gain \times BDP$
- 发送速率: $pacing_rate = pacing_gain \times BtlBw$
- $cwnd_gain$ 为拥塞窗口增益系数
- $pacing_gain$ 为发送速率增益系数

在稳态，BBR交替进行带宽探测和延迟探测，98%时间用于带宽探测

■ $RTprop$ 和 $BtlBw$ 的探测过程

包括四个阶段

- 启动阶段
- 排干阶段
- 探测带宽阶段
- 探测RTT阶段



4.2.4 TCP拥塞控制：DCTCP

■ 主要针对数据中心网络（DCN）的特点而设计

- DCN通常使用低成本白盒交换机（路由器）进行构建，其缓冲区比较小，对于突发流量更容易产生丢包，丢包的恢复会产生较大的延时
- DCN通常存在大流和小流，大流对缓冲区的占用会导致小流经历较大的延时
 - 大流：带宽消耗大、传输时间长的数据流
 - 小流：传输数据量小、传输时间短的数据流

■ 利用显示拥塞通知（ECN）对TCP拥塞控制机制进行扩展

4.2 传输控制协议



4.2.4 TCP拥塞控制：DCTCP

■ DCTCP的拥塞控制

- 当IP数据包到达路由器时，如果缓冲队列长度大于阈值 K ，则设置数据包首部中的**CE**标志，即将IP数据包首部中服务类型与拥塞通知字段的后两位设置成11
- **接收方**为每个TCP连接维护一个布尔变量*ReceivedCE*，初始值为False
 - 当*ReceivedCE*为True时，发送的ACK段首部中的**ECE**标志置位
 - 当接收到设置**CE**标志的IP数据包时，如果*ReceivedCE*为False，将其设置为True，立即发送ECE标志置位的ACK段
 - 当接收到未设置**CE**标志的IP数据包时，如果*ReceivedCE*为True，将其设置为False，并立即发送ACK段
 - 其他情况忽略CE标志

4.2 传输控制协议



4.2.4 TCP拥塞控制：DCTCP

■ DCTCP的拥塞控制

- **发送方**根据ECE标志置位的ACK段的比例调整拥塞窗口的大小，每接收到一个ECE标志置位的ACK段，按如下公式调整拥塞窗口：

$$cwnd = cwnd \times (1 - \alpha/2)$$

$$\alpha = (1 - g) \times \alpha + g \times F$$

- g 是可以调整的参数 ($0 < g < 1$)
- F 是上一个RTT内接收到ECE标志置位的ACK段的比例

- **优点**：DCTCP能够保持路由器的缓冲区具有较低的占用率，为延迟敏感的小流提供较低的延迟，并提高对高突发流量的容忍，且对大流保持较高的吞吐量

4.1 用户数据报协议

4.2 传输控制协议

4.3 TCP扩展与替代协议

4.3.1 MPTCP

4.3.2 QUIC

4.4 实时传输协议

4.3 TCP扩展与替代协议



TCP面临的性能问题

■ TCP连接不支持多路径传输和网络接口之间的动态切换

- TCP使用四元组<源IP地址, 源端口, 目的IP地址, 目的端口>标识一个TCP连接, 其中任何一元发生改变, 连接将不能持续。因此, 一个TCP连接只能使用主机中的一个IP地址
- 数据中心的服务器、移动终端都配有多多个网络接口, 一个TCP连接不能利用多个网络接口进行多路径传输, 也不支持不同网络接口之间的动态切换, 不能对负载均衡和移动性提供良好的支持
 - 例如, 从WiFi切换到5G, 或从5G切换到WiFi



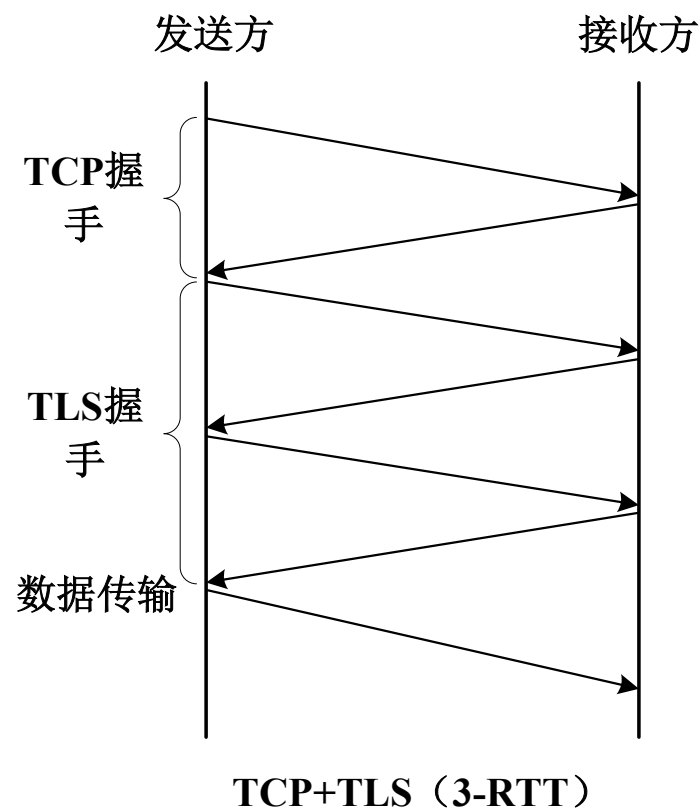
4.3 TCP扩展与替代协议



TCP面临的性能问题

■ TCP叠加安全传输层协议（TLS）会引入较大的握手延时

- 随着应用对数据保密性和完整性的需求，目前绝大多数Web服务器都支持HTTPS，即在HTTP和TCP之间引入了TLS层
- TCP叠加TLS在握手期间至少需要3个RTT，这对HTTP的性能会产生较大的影响
 - 特别是长肥管道及网络连接频繁切换的移动环境
- 其他使用TLS和TCP服务的应用也会产生类似的问题



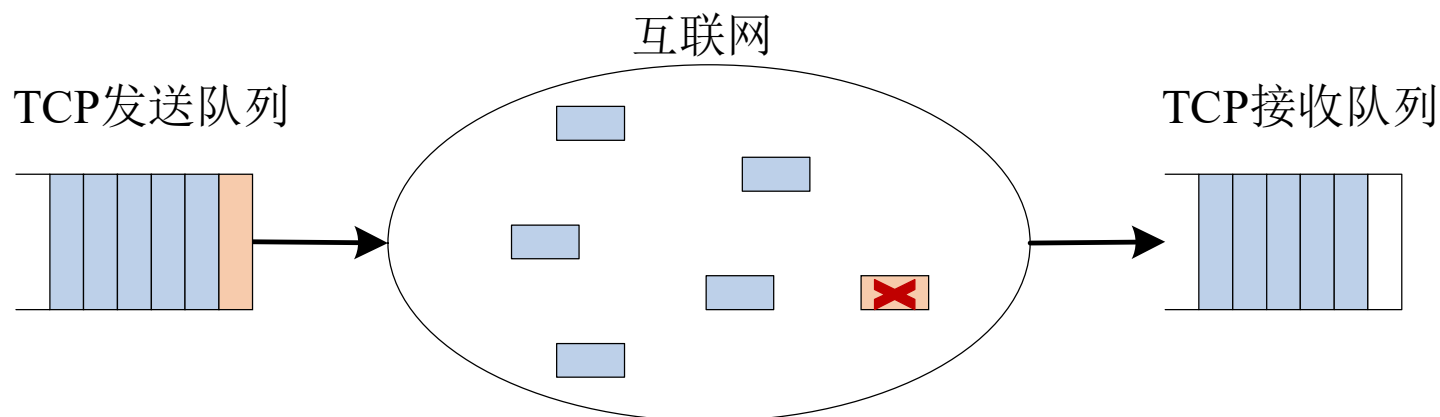
4.3 TCP扩展与替代协议



TCP面临的性能问题

■ TCP的确认-重传机制及按序交付服务会形成队头阻塞

- TCP需要按序将字节流交付给应用进程，某个TCP数据段丢失，需要采用重传机制进行恢复，超时重传机制会引入较大的延时
- 丢失数据没有到达之前后面的数据均不能交付给应用进程，由此产生**队头阻塞问题**，队头阻塞问题会对应用的性能产生较大的影响
 - 在HTTP应用中，在需要同时传输多个元素时，可以同时建立多个TCP连接来缓解TCP队头阻塞问题，但也引入了额外的资源开销



4.3 TCP扩展与替代协议



4.3.1 多路径TCP (MPTCP)

- **目标**: 对TCP进行扩展, 使TCP连接支持多路径传输和网络接口之间的动态切换, 并保持良好的兼容性
- **MPTCP设计原则**
 - **应用程序的兼容性**: 基于TCP的应用程序不需要做任何修改, 可以直接在MPTCP上运行
 - **网络的兼容性**: 能在现有的互联网结构中正常运行, 不需要对网络中的路由器、防火墙、NAT等中间设备进行任何改变
 - **传统TCP协议的兼容性**: 通过TCP选项实现, 可以进行MPTCP能力协商, 不支持MPTCP能力的主机, 可以使用传统的TCP

4.3 TCP扩展与替代协议



4.3.1 多路径TCP (MPTCP)

■ MPTCP在TCP/IP协议栈中的位置

- 向上提供与TCP相同的编程接口，应用程序不需要进行任何修改
- 对于IP层，每个MPTCP子流就是常规的TCP连接
- MPTCP负责开启和关闭子流，将单一数据流切分为若干个子流，同时利用多条路径进行传输

应用层	
MPTCP	
子流 (TCP)	子流 (TCP)
IP	IP

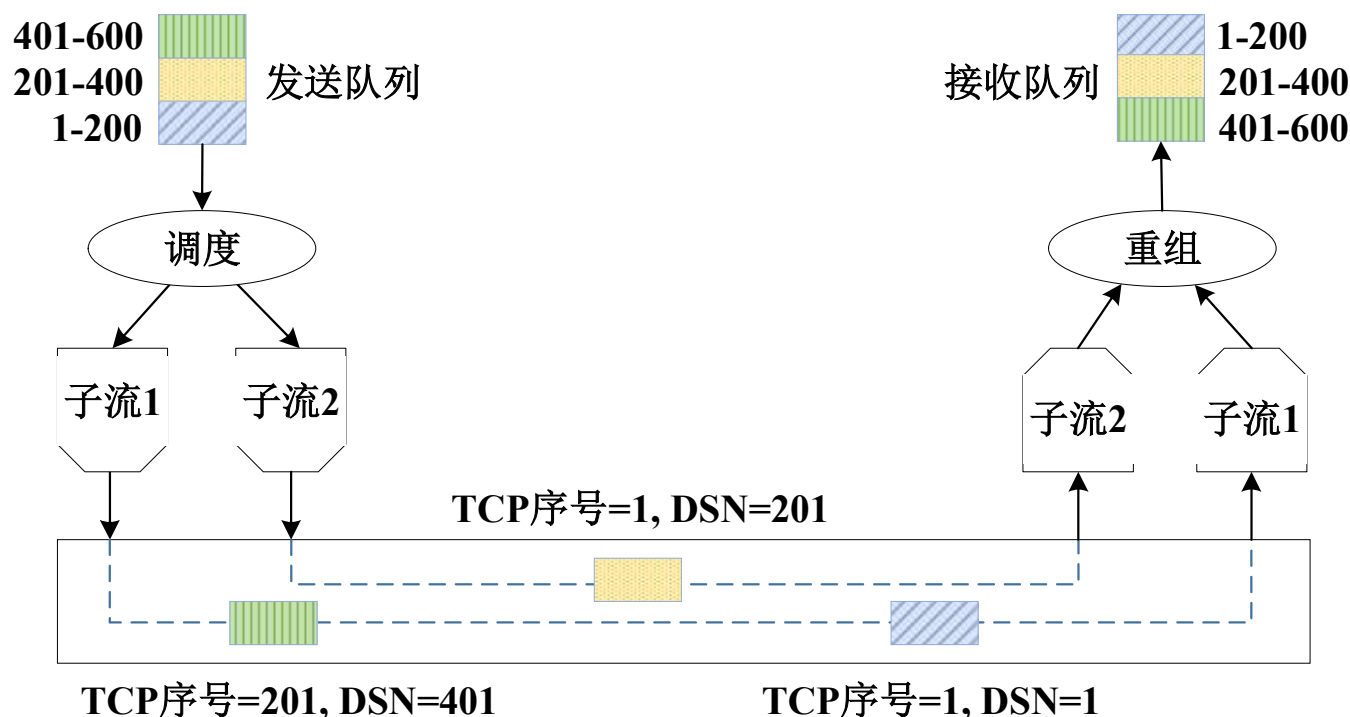
4.3 TCP扩展与替代协议



4.3.1 多路径TCP (MPTCP)

■ 数据传输示例

- 发送方基于某种算法将发送队列中的数据调度到两个子流进行传输
- 接收方基于TCP选项中**子流序号**和**数据序号**之间的映射关系对子流的数据进行合并，形成与发送顺序一致的单一数据流交付给应用进程



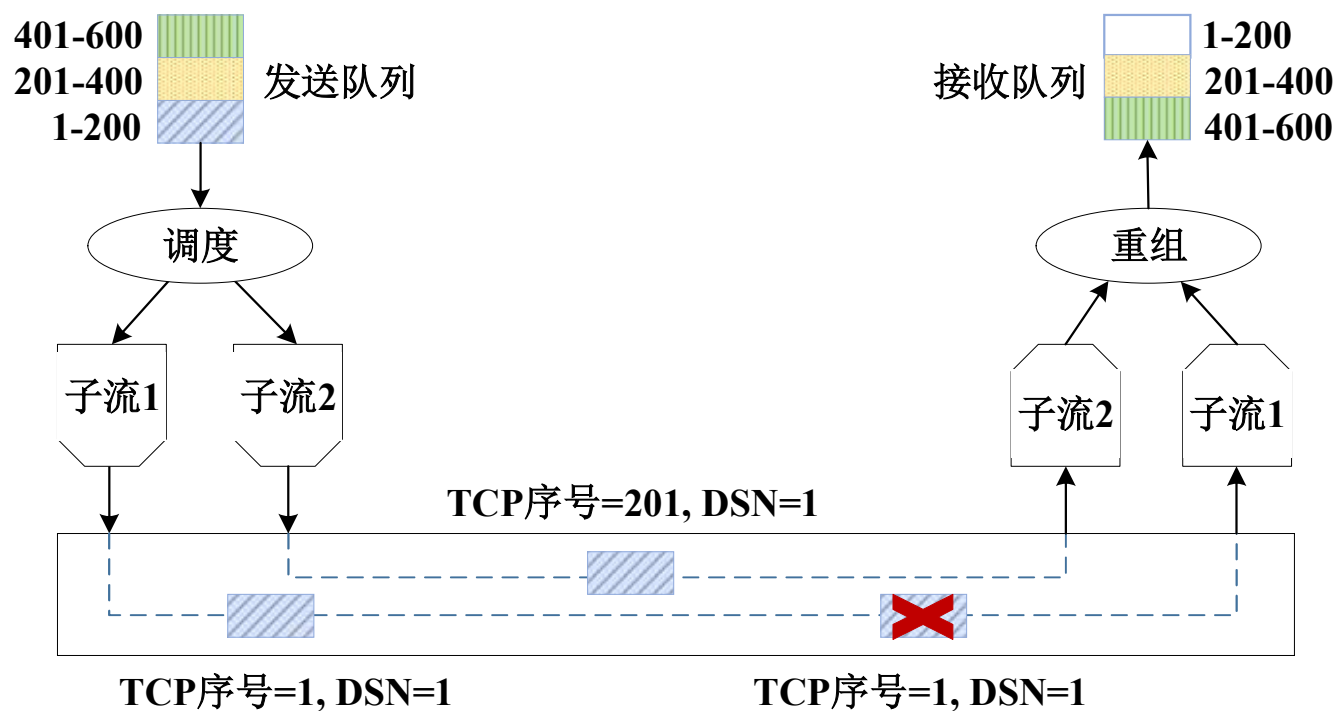
4.3 TCP扩展与替代协议



4.3.1 多路径TCP (MPTCP)

■ 数据传输示例：重传

- 重传数据段可以被调度到任意子流上进行传输，不必与第一次传输使用相同的子流



4.3 TCP扩展与替代协议

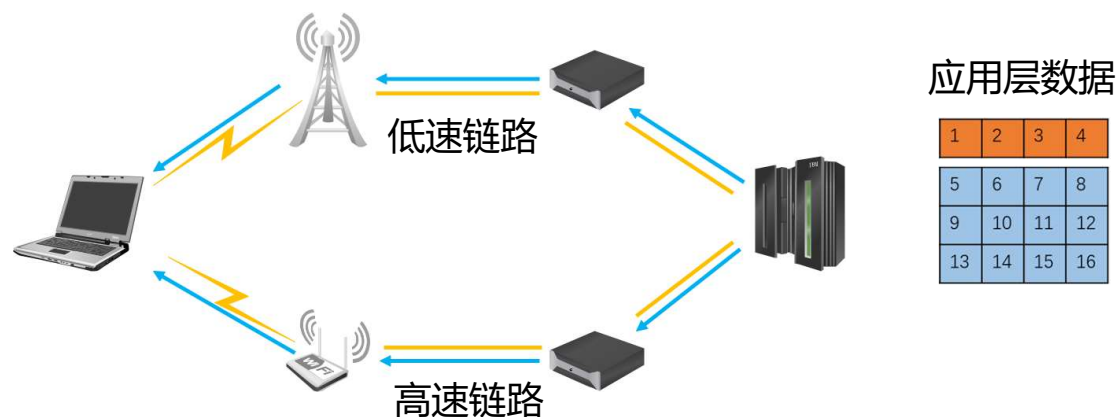


4.3.1 多路径TCP (MPTCP)

■ 应用场景及支持

- 移动智能终端的WiFi、5G等多连接
- 数据中心网络的负载均衡)
- iOS 操作系统、三星手机均支持MPTCP
- Linux内核支持MPTCP，当前最新版本为v0.96

■ 多路径传输示例



4.3 TCP扩展与替代协议



4.3.2 QUIC

■ QUIC是由谷歌公司设计的一种端到端、安全可靠传输协议

- 与TCP类似，主要实现可靠传输、流量控制、拥塞控制等功能
- 2021年5月形成标准文档RFC 9000
- 目前得到了许多服务器和浏览器的支持，在数据中心网络和运营商网络中得到了应用

■ **主要动机：**解决TCP面临的性能问题

- 移动性和连接迁移问题
- 握手时间过长问题
- 队头阻塞问题

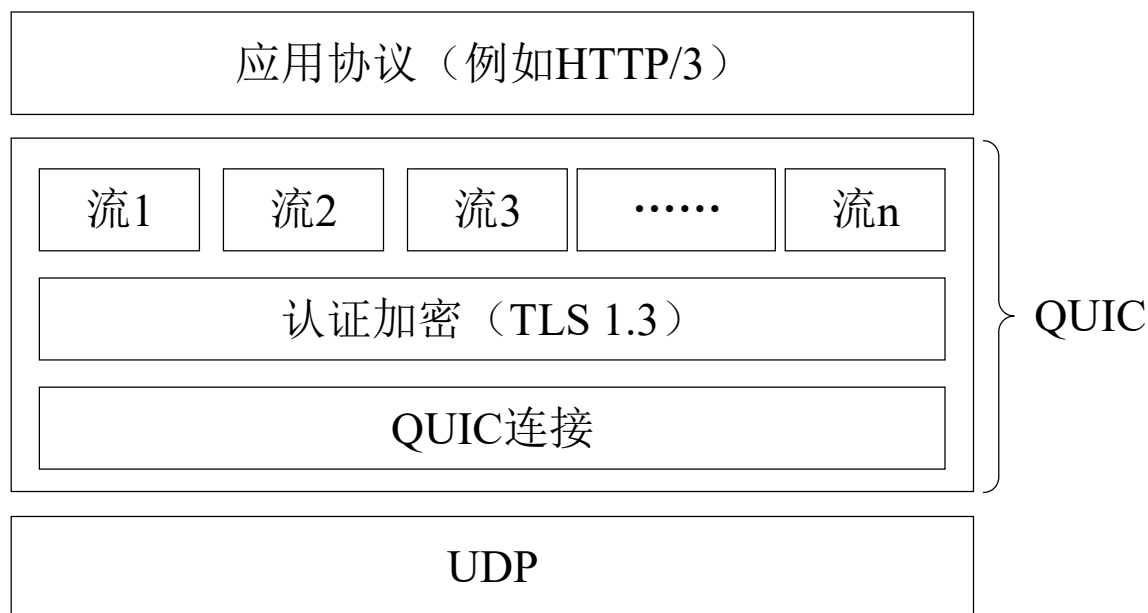
4.3 TCP扩展与替代协议



4.3.2 QUIC

■ 在TCP/IP协议栈中的位置

- 基于UDP提供的服务，在操作系统的用户空间（用户态）实现
 - QUIC可以像TCP和UDP一样直接在操作系统内核中实现，但是操作系统的更新速度往往跟不上应用的需求
 - 在用户空间实现，相较于内核空间实现的迭代速度更快



4.3 TCP扩展与替代协议



4.3.2 QUIC

■ QUIC连接

➤ QUIC是一种面向连接的可靠传输协议

- 使用连接标识符（Connection ID）标识连接
- 连接标识符由两端独立选取，与连接端点的IP地址或UDP端口号解耦
- 连接端点的IP地址或UDP端口号发生变化时，连接标识符仍然能识别原有的连接，可以方便地实现QUIC连接在不同网络接口或UDP端口之间的**动态迁移**

➤ 内嵌TLS，将TLS握手和传输握手进行组合，最小化连接建立延时

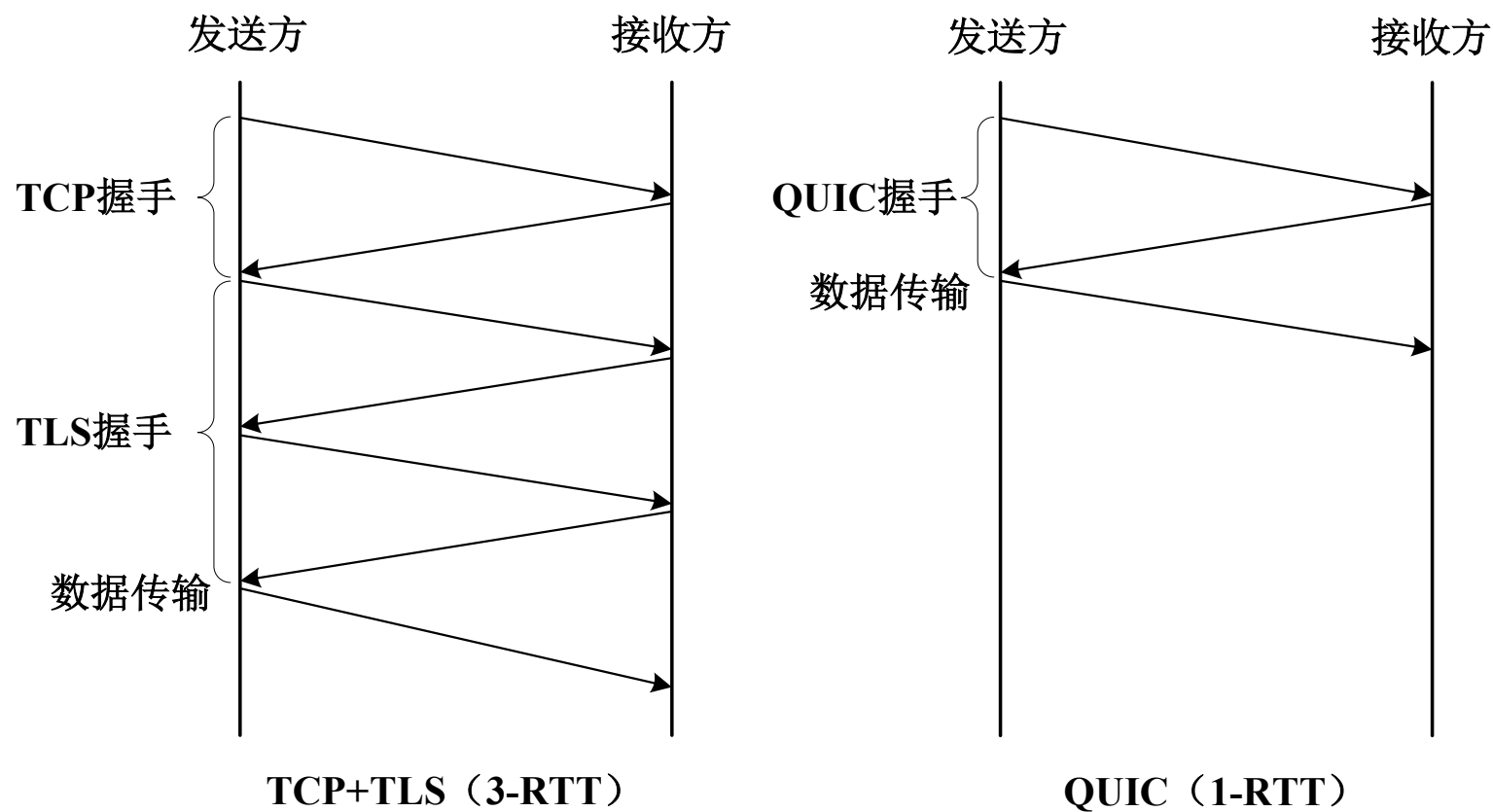
- TCP加TLS的握手需要3个RTT，在RTT达到数百毫秒时，会严重影响用户的体验
- QUIC可以将握手过程减少到1个RTT或0个RTT，**连接延时显著降低**

4.3 TCP扩展与替代协议



4.3.2 QUIC

■ TCP+TLS与QUIC握手过程对比



4.3 TCP扩展与替代协议



4.3.2 QUIC

■ 流机制与连接复用

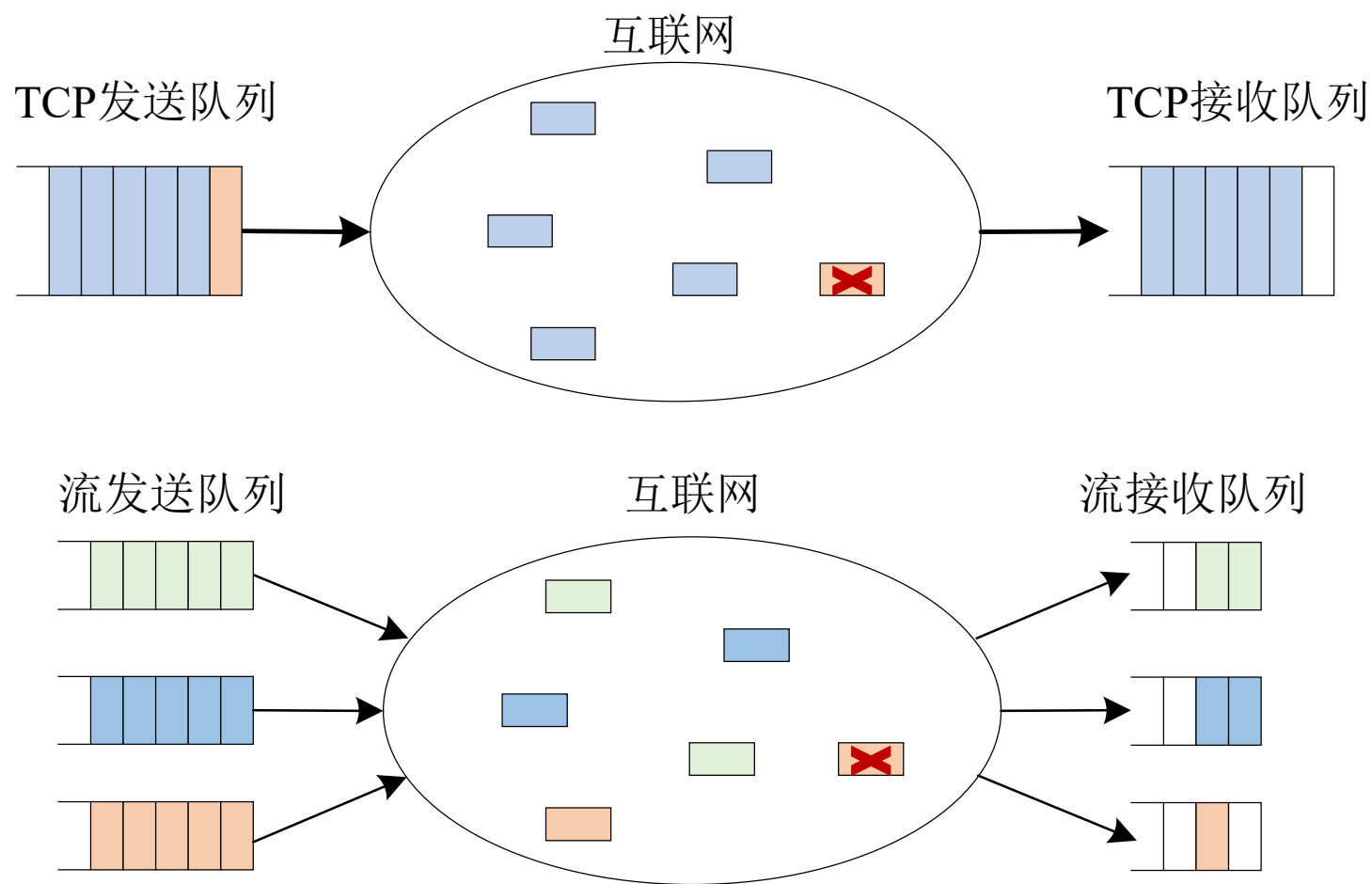
- 在单一QUIC连接上可以建立多个流，每个流由流标识符（Stream ID）进行标识
 - 64位流标识符，单增使用，不重复
 - 为了避免冲突，客户发起的流使用偶数流标识符，服务器发起的流使用奇数流标识符
- 流为应用提供了轻量级、有序的字节流抽象
 - 发送方将字节流封成帧，接收方将帧重组成有序的字节流
 - 多个流的帧可以混合组装成分组进行发送
 - 可以保证单个流数据的有序交付，流之间的数据不存在顺序依赖关系，某个流中的数据丢失不会影响其他流的交付，可以缓解队头阻塞问题

4.3 TCP扩展与替代协议



4.3.2 QUIC

■ 解决队头阻塞问题示例



4.1 用户数据报协议

4.2 传输控制协议

4.3 TCP扩展与替代协议

4.4 实时传输协议

4.4 实时传输协议（RTP/RTCP）



- RTP/RTCP是由IETF制定的，用于支持实时数据传输的端到端传输协议，对应的文档为RFC3550
 - RTP：负责实时数据传输（数据流）
 - RTCP：负责监控服务质量并提供控制信息（控制流）
- RTP/RTCP设计目标
 - 主要为多方参与的多媒体会议而设计（也适用于其它实时应用）
 - 允许灵活扩展会话的规模，一个会话可以支持几个到几千个参与者
- RTP/RTCP应用
 - 当今主流的实时传输协议之一，被实时多媒体应用广泛使用，如WebRTC、RTSP等
 - 典型应场景：音视频会议、实时直播等

4.1 用户数据报协议

4.2 传输控制协议

4.3 TCP扩展与替代协议

4.4 实时传输协议